

### Основни понятия

**Хипертекст** – текст, съдържащ връзка към друга информация.

**Хипервръзка (hyperlink)** – връзка в хипертекст към друга информация (хипертекстов документ)

**Уеб сървър** – мястото, на което са разположени уеб страниците, групирани в един уеб сайт.

**HTTP** – протокол, който се използва за обмен на хипертекстове.

**Архитектура клиент – сървър** – начин на организация на работа и обмен на данни в интернет.

**Уеб стандарти** – нормативни спецификации на технологии и методологии за визуализиране на съдържание в уеб среда.

### Речник

**HTTP (Hypertext Transfer Protocol)** – протокол за трансфер на хипертекстове

### Защо трябва да спазвате стандартите?

- Опростяване на процеса на разработка, по-лесна поддръжка.
- Бъдеща съвместимост.
- По-бързо зареждане и визуализиране на страниците.
- По-добра достъпност и възможности.
- Намаление на разходите.
- По-добри резултати в търсачките.
- Лесна адаптация.
- Улесняване на потребителите.

### Въпроси и допълнителни задачи

1. Какво е основното предназначение на хипертекста?
2. За какво се използват уеб стандартите?
3. Отворете интернет адреса <https://www.w3.org/standards/> и се запознайте със стандартите, които предлага организацията W3C.
4. Проучете предназначението на езика **JavaScript**. Създайте текстов документ, като опишете в него наученото.

### Организация на работа в уеб пространството

Основният метод за предаване на информация в интернет, при който части от текст, изображение, звук и др. могат да съдържат връзка към друга информация, се нарича **хипертекст**. Самата връзка се нарича **хипервръзка (hyperlink)**.

Информацията в интернет е организирана в уебстраници, които са хипертекстови документи. Те могат да съдържат текст, изображения, видео и други компоненти. Езикът, използван за описване на това съдържание се нарича **HTML (HyperText Markup Language)**.

Всяка страница има свой собствен адрес (URL), който не се повтаря. Уеб страниците са групирани в уеб сайтове, които са съвкупност от страници, допълващи се функционално, имат сходен дизайн и обикновено са разположени на един **уеб сървър**. Сайтовете се съхраняват на сървъра под формата на файлове. Всеки уеб сайт има собствено уникално име, по което да бъде различаван в интернет.

Организацията на работа и обменът на данни в интернет се основават на **мрежовата архитектура клиент – сървър**. От едната страна са клиентите (уеб браузъри), които подават заявки и изчакват отговор, а от другата страна са сървърите (компютри, съхраняващи огромен капацитет от ресурси), които чакат заявки, за да ги обработят, и предлагат своите услуги на клиентите.

Обмен на хипертекстове (WWW) е най-често използваната услуга в интернет. Протоколът, който се използва за този обмен, се нарича **HTTP (Hypertext Transfer Protocol)**. Той осъществява връзката между клиента (уеб браузър) и сървъра (уеб сървър). Уеб браузърът изтегля и изобразява страниците от уеб сървъра, където са съхранени.

### Уеб стандартите и тяхното предназначение

Интернет е създаден през 70-те години на миналия век. От тогава насам мрежата се развива непрекъснато. Използват се нови технологии, добавят се нови функционалности. Това налага въвеждането на стандарти. Без тези стандарти например всеки браузър ще решава самостоятелно как точно да показва информацията. Уеб страницата може да изглежда напълно различно в различните браузъри. Затова възникват **уеб стандартите**.

**Уеб стандартите** представляват нормативни спецификации на технологии и методологии за визуализиране на съдържание в уеб среда. Те са необходими при структуриране на информацията, която се съхранява и се показва на потребителите.

**World Wide Web Consortium (W3C)** е главната международна организация за стандартизация в интернет. Тя се занимава със задълбочено изследване на видовете публикувано уеб съдържание, като целта ѝ е да осигури и въведе стандартна система за разработване, публикуване и разпространение на съдържанието в интернет. W3C е основана от Тим Бърнърс-Лий през октомври 1994 г. в лабораторията по компютърни науки на Масачузетския технологичен институт в САЩ.

Уеб дизайн стандартите се отнасят до това как една страница/сайт трябва да бъде проектирана. Ето някои примери за стандарти за уеб дизайн, които се отнасят до външния вид на уебсайт:

- кои шрифтове да се използват, с какви размери;
- кои цветове да се използват и къде;
- стил на бутоните, върху които може да се кликва;
- разположение на менюто за навигация (стандартите за уеб дизайн казват, че трябва да е в горната част на страницата);
- къде да се постави функцията за търсене.

Уеб дизайн стандартите обхващат както естетическия външен вид, така и функционалното устройство на сайта. С навлизането на мобилните устройства се появява нуждата сайтовете да се виждат на тях (mobile friendly). Появява се т.нар. адаптивен дизайн:

**Адаптивен дизайн.** Адаптивният дизайн, наричан още респонсив (responsive), е предназначен за визуализиране на сайтове на дисплеи с всякакъв размер. Неговото изработване включва в себе си стилна, визуална и гъвкава подредба на всички елементи по специфичен начин, така че в крайна сметка външният вид и структурата на страницата да бъдат достатъчно удобни, достъпни и прегледни за всички интернет настолни или мобилни устройства. Това става посредством подбрани HTML компоненти със специфични CSS стилове.

#### • Уеб стандартите са важни за поддръжката.

Използването на правилните инструменти е част от всеки проект. Решаването на повечето проблеми при уеб сайтовете става чрез максимално придържане към уеб стандартите в рамките на всеки проект. Това гарантира, че уеб сайтът ще се нуждае от минимални усилия по поддръжката за срока на своето използване.

#### • Уеб стандартите са важни за използваемостта на уеб сайта.

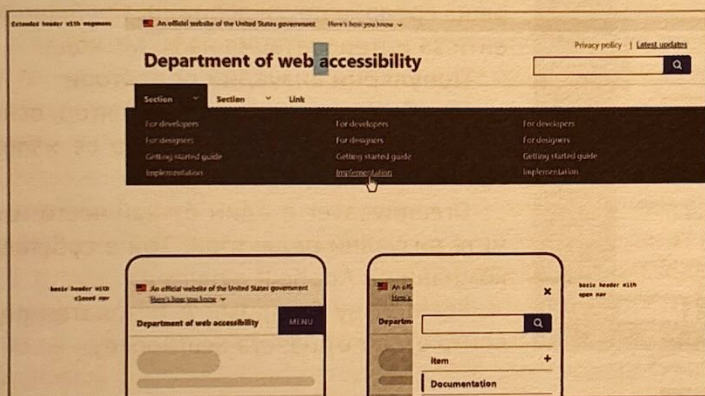
Използването на уеб стандарти в дизайна ще гарантира, че сайтът ще бъде привлекателен за по-голямата част от хората, които го посещават. Ние не просто искаме да направим сайта достъпен за един браузър, ние трябва да се уверим, че уеб сайтът работи във всички браузъри.

#### • Уеб стандартите правят поддръжката лесна.

#### • Уеб стандартите означават качество на уеб дизайна.

Да използваш уеб стандарти, означава да държиш на качеството на това, което правиш.

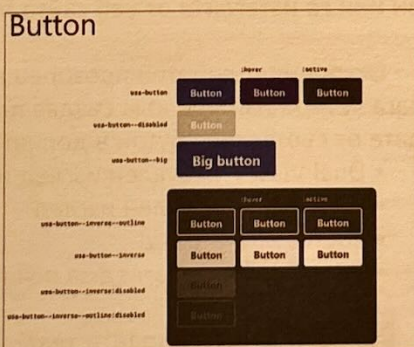
Много големи организации също се опитват да наложат свои стандарти за уеб дизайн, например правителства, автомобилни концерни и др. На фиг. 1 и фиг. 2 са дадени части от публикуваните стандарти на правителството на САЩ.



Фиг. 1

**Задача.** Сравнете стандартите за уеб дизайн на правителството на САЩ от два различни периода и вижте какви са промените.

За целта отворете файловете от папка "Standarts" от приложените материали. Потърсете първо еднакви уеб компоненти – напр. Бутони, Съобщения (Alerts) и др. Освен промените в компонентите, има ли други разлики?



Фиг. 2

## Основни уеб технологии, използвани в интернет

Изборът на уеб технологии при изграждане на сайтове в интернет е част от процеса на планиране на даден проект. Всеки уеб сайт има нужда от правилно избрани технологии за успешната си реализация. Основните фактори, които влияят при избора на технология, са честота и обем на актуализациите в сайта.

## Видове уеб технологии

• **HTML** (HyperText Markup Language) е основният маркиращ език за описание и дизайн на уеб страници. Чрез него се описва съдържанието на уеб сайта, което браузърът преобразува в екранно изображение. Правилата за използването на HTML са стандартизирани от международния консорциум W3C.

• **CSS** (Cascading Style Sheets) е език за описание на стилове, който работи с HTML и указва как точно да се визуализира всеки HTML компонент. CSS се превежда като "каскадни стилови набори", но често се използва и терминът "таблицы със стилове". Всеки стил набор съдържа съвкупност от стилове, с които се описва оформление на страницата и отделните ѝ елементи (абзаци, таблици и др.). Чрез стилове се задават цветът на текста, подравняването на абзаца, отстъпите между изображенията, рамките и др.

• **JavaScript** е език за програмиране, който обикновено се използва за създаване на интерактивни ефекти в уеб браузера. Чрез него се дава достъп до различни части на дадена уеб страница, като например форма, изображения, целия документ или дори прозорец на браузъра. С JavaScript се създават скриптове (scripts – сценарии) – това са минипрограми, които могат да контролират, променят и преобразуват различни обекти върху страницата.

• **PHP** (Hypertext Preprocessor) е скриптов език, който се използва за уеб програмиране при създаване на сървърни приложения и динамично уеб съдържание. Той е един от най-популярните езици за програмиране в интернет. Разпространява се под отворен лиценз и позволява безплатно разпространяване на програмния код на интерпретатора на езика.

• **XML** (eXtensible Markup Language) е маркиращ език, който се използва като универсално средство за предаване на данни в мрежи. С помощта на тагове се описва структурата на данните, а вътрешно между таговете е съдържанието.

• **SQL** (Structured Query Language) е език за програмиране, предназначен за създаване, изменение, извличане и обработване на данни от релационни системи за управление на бази данни.

• **CGI** (Common Gateway Interface) е стандарт на интерфейс, предназначен за съвместна работа на уеб сървър и интерпретаторите на скриптове, който осъществява връзка между външна програма и уеб сървъра посредством HTTP протоколи. Съвместната работа на програма с уеб сървър води до приемане на параметрите, подадени от клиента, прехвърляне към програмата, която генерира резултат, и връщане на този резултат обратно на клиента. CGI се използва за разработка на търсещи системи, чатове, търговски обяви, книги за гости, за изпращане на поща, изработка на банери и др.

### HTML редактори за разработка на уеб сайт

Разработката на уеб сайтове се осъществява с помощта на HTML редактори. HTML редакторът е софтуерно приложение за създаване на уеб страници. По принцип HTML кодът на една уеб страница може да бъде написан с всеки текстов редактор, но съществуват специализирани HTML редактори, които улесняват създаването на страницата, като предлагат удобство и по-добра функционалност.

HTML редакторите се делят на текстови и визуални.

**Текстовите редактори** се използват за ръчно въвеждане на HTML код от специалистите. Най-често използваните са Notepad++, Bluefish Editor, UltraEdit, EmEditor. Тези редактори познават синтаксиса и структурата на HTML и свързаните с него езици (JavaScript, CSS, XML и др.), улесняват писането на кода и го правят по-четивен посредством подходящо оцветяване.

**Визуалните редактори** реализират технологията WYSIWYG (What You See Is What You Get) – каквото виждате, това получавате. При нея уеб страницата се разработва с помощта на визуална платформа и авторът вижда резултата от своите действия. WYSIWYG означава, че завършената

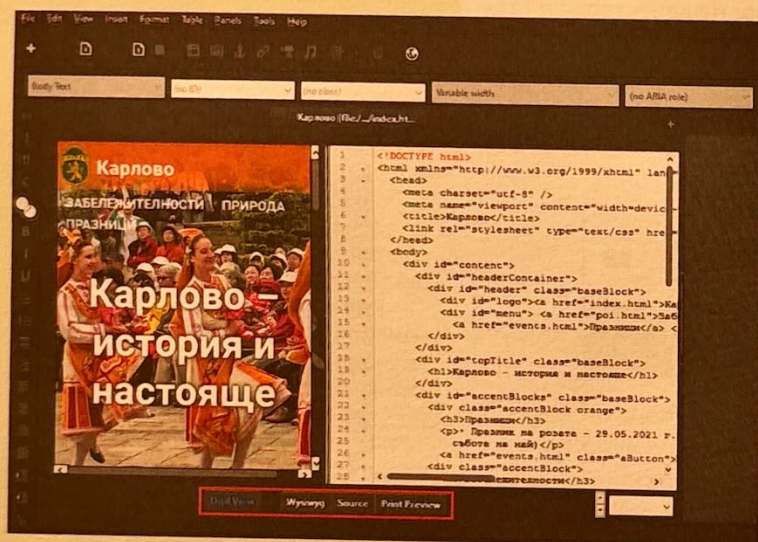
уеб страница ще изглежда точно по същия начин, както е била проектирана. Основното предимство на визуалните редактори е, че не е необходимо авторите да познават езика HTML, тъй като кодът се генерира автоматично. Някои визуални редактори имат собствена интерпретираща среда, а други асоциират външни уеб клиенти за интерпретация на HTML кода.

#### Популярни визуални редактори:

BlueGriffon е безплатен редактор, основан на базата на ядрото Gecko, който се използва от браузъра Mozilla Firefox.

Dreamweaver е един от най-често използваните визуални редактори. Той е собственост на компанията Adobe и е платен.

SeaMonkey Composer е безплатен редактор, който е част от пакета SeaMonkey.



Фиг. 1

**Задача 1.** Инсталирайте HTML редактора BlueGriffon, като използвате инсталационния файл от папката "Software" от приложените материали или го изтеглите от уеб сайта <http://www.bluegriffon.org/#download>.

Отвореният работен прозорец на редактора BlueGriffon (фиг. 1) предлага четири изгледа към създаваната страница, които можете да разгледате от съответния бутон в долната му част:

- Dual View – визуален изглед и HTML код
- Wysiwyg – визуален изглед
- Source – HTML код
- Print Preview – изглед за печат

**Задача 2.** Инсталирайте текстовия редактор Notepad++, като използвате инсталационния файл от папка "Software" от приложените материали или го изтеглите от уеб сайта <https://notepad-plus-plus.org/>.

**Задача 3.** Отворете файла pois.html от папката "site" от приложените материали в двата редактора.

• В редактора BlueGriffon влезте в режим Dual View и пробвайте да изберете различни обекти от лявата (визуалната) част. Виждате как в дясната част се осветява съответният код. Всъщност това е HTML кодът, който браузърът визуализира като страница.

- работата в този режим (Dual View) не е много препоръчителна, особено ако искате да пишете HTML код. Затова, когато ще въвеждате код, използвайте режима Source или редактора Notepad++.
- Сравнете визуално как изглежда кодът в двата редактора. Кой от двата редактора ще използвате за следващите упражнения, е въпрос на личен избор.

## HTML шаблони

**HTML шаблоните** (templates) са предварително създадени уеб страници или части от уеб страници. Те могат да се използват многократно за създаване на страници с подобно съдържание, като това улеснява процеса по създаването на уеб сайтове. Шаблоните са лесни за модифициране, спестяват време и разходи.

HTML шаблоните могат да съдържат горен и долен колонтитул, съдържание, лого, различни компоненти (панели, изображения) и елементи. Те са разделени на категории, видове и стилове.

HTML шаблоните се редактират с помощта на визуални редактори. В интернет има както платени, така и безплатни шаблони. Съществуват и уеб шаблони на сайтове за системите за управление на съдържанието, като WordPress, Joomla и др.

**Задача 4.** Изтеглете безплатен HTML шаблон от интернет сайта [www.free-css.com](http://www.free-css.com). Стартирайте редактора BlueGriffon и от менюто File/Open File отворете файл с разширение .html в режим Wysiwyg. Редактирайте неговото съдържание.

## Авторски права и лицензи при използване на HTML редактори и шаблони

При работа с HTML редактори и използването на HTML шаблони се изисква да зачитате правата на авторите им и да посочвате източниците си на информация.

Нетърговската организация **Creative Commons (Криейтив Комънс, превод от англ. – Творчески общности)** е създала безплатни за ползване типове договори – свободни и несвободни публични лицензи. С тяхна помощ авторите и държателите на авторски права могат да дават на потребителите по-широки права за свободно ползване на произведенията си от тези, които им предоставя авторското право. Целта на Creative Commons е да се увеличи броят на произведенията, достъпни за легално разпространение и използване от потребителите. Договорите Creative Commons ви дават възможност свободно да копирате, разпространявате, публично да представяте или да използвате произведението с нетърговски цели (фиг. 2).

| Мога да                    | Само ако                                                    |
|----------------------------|-------------------------------------------------------------|
| споделям, променям, печеля | спомена автора                                              |
| споделям, променям, печеля | спомена автора и запазя лиценз                              |
| споделям, променям         | спомена автора и не печеля                                  |
| споделям и печеля          | спомена автора и не променям                                |
| споделям, променям         | спомена автора, запазя същия лиценз, не печеля              |
| споделям                   | спомена автора, не променям, не печеля, запазя същия лиценз |

Фиг. 2

**GNU General Public License** (превод от англ. – **Общ публичен лиценз на ГНУ**) е лиценз на свободен софтуер, по който авторът предава програмното обезпечение в обществена собственост. GPL гарантира на потребителите на компютърни програми определени права: да ползват програмата за каквато и да е цел; да изучават как работи програмата и да я променят; да разпространяват нейни копия; да подобряват програмата и да дават на обществото достъп до подобренията.

## Сайтове, съдържащи безплатни шаблони:

- [www.free-css.com](http://www.free-css.com)
- [www.wix.com](http://www.wix.com)
- [nicepage.com](http://nicepage.com)

## Въпроси и допълнителни задачи

1. Какво е основното предназначение на HTML редакторите?
2. Какво представлява **GNU General Public License**?
3. Изтеглете безплатен HTML шаблон и го редактирайте с BlueGriffon.

Услугата World Wide Web представлява мрежа от свързани онлайн документи, създадени с **HTML** (англ. HyperText Markup Language – език за маркиране на хипертекст). HTML е основният език за описание на уеб страници.

### История

Езикът HTML е разработен от британския учен Тим Бърнърс-Лий в периода 1986 – 1991 г. в организацията ЦЕРН в Женева (Швейцария). Той се създава като език за обмен на научна и техническа документация. Той се назначен за използване от хора, които не са специалисти в областта на оформлението. Първият публикуван документ е от края на 1991 г., наречен "HTML tags", в който се описват 18 елемента, съставляващи първоначален и създаден като средство за структуриране и форматиране на документи, без да е обвързан със средства за възпроизвеждане (показване). С течение на времето се добавят нови функционалности, свързани с използването на мултимедия и графичен дизайн. От 2000 г. HTML се превръща в световен стандарт. От 2014 г. до днес се използва версията HTML5.

Създаването на уеб страница с помощта на HTML може да се извърши с обикновен текстов редактор (Notepad, Notepad++) или специализирани приложения за създаване на HTML страници (BlueGriffon, Dreamweaver), като се записва в текстови файлове с разширение .html или .htm.

Описанието на HTML документ се състои от специални елементи, наречени **тагове**.

### Тагове

Тагът (англ. tag – етикет) се състои от текст, заграден в ъглови скоби (например <html>), и указва на браузъра как да интерпретира последващата информация, за да бъде показана на дисплея на устройството.

Всеки таг има конкретно предназначение. Таблица с всички възможни тагове и тяхното предназначение можете да видите на адрес [www.w3schools.com/tags/](http://www.w3schools.com/tags/) или като отворите файла "Web\HTML\HTML Tags.pdf" от приложените файлове.

Всеки HTML елемент, с малки изключения, се описва по следния начин:

– таг за начало (отварящ) – маркира началото на HTML елемента, например <html>;

– съдържание;

– таг за край (затварящ) – маркира края на HTML елемента; за разлика от началния таг крайният има наклонена надясно черта пред името си, например </html>.

Ето как изглежда цялата комбинация:

```
<html> ... съдържание ... </html>
```

Таговете могат да се влагат един в друг, като първият отворен таг се затваря последен.

#### Атрибути

Към много от таговете могат да се добавят атрибути (променливи), които съдържат допълнителна информация за тага. Например в тага за изображение <img>

```

```

има атрибути с имена src, width, height, които имат следните стойности:

```
src="img_girl.jpg"
width="500"
height="600"
```

Атрибутите могат да бъдат задължителни и допълнителни, като това

### Основни понятия

**HTML** (HyperText Markup Language) – стандартен маркиращ език за създаване на документи, които да се визуализират в уеб браузър.

**HTML елемент** – основен структурен компонент в HTML документа. Описва се с отварящ и затварящ HTML таг и включва цялото съдържание между тях.

**HTML таг** – етикет, заграден в ъглови скоби (< >). Таговете указват на браузъра как да интерпретира информацията, описана в HTML документа, като определят началото и края на описанието на HTML елементите, както и тяхното съдържание, вид и атрибути.

**HTML атрибут** – задава основни или допълнителни характеристики на HTML елемента. Описва се в отварящия таг, най-често като двойка *атрибут=стойност*.

зависи от съответния таг. В горния пример тагът `<img>` изисква задължителен атрибут `src`, а атрибутите `width` и `height` могат и да не се задават. Всеки таг поддържа само определен набор от атрибути – т.е., ако добавим към тага други атрибути, те няма да се възприемат. Стойността на атрибутите се огражда в кавички (" ").

#### Пример:

използване на атрибута за хипервръзка `href`

```
<a href="https://www.google.bg/"> ..... </a>
```

Списък на атрибутите, които се поддържат в HTML таговете, можете да видите на адрес [www.w3schools.com/tags/ref\\_attributes.asp](http://www.w3schools.com/tags/ref_attributes.asp) или като отворите файла "Web\HTML\HTML Atributes.pdf" от приложените материали.

## Структура на HTML документ

HTML документите са съставени от две основни части: глава (**head**) и тяло (**body**).

HTML документът трябва да започва с декларацията `<!DOCTYPE html>`. Това не е HTML таг, а инструкция за уеб браузъра – прилежащите му атрибути определят версията на HTML езика и какъв тип е документът. Ако документът е написан на актуалната версия на езика (HTML5), не се задават аргументи.

Основните тагове, които оформят HTML документ, са:

- **<html>**

Указва на браузъра, че това е HTML документ. Отбелязва началото и края на документа и съдържа всички други негови елементи. Въвежда се след началния таг `<!DOCTYPE html>`.

- **<head>**

Съдържа заглавната част на файла и информация за идентифициране на HTML документа. Между неговото начало и край се поставя тагът за заглавие на документа.

- **<title>**

Съдържа заглавието на документа. Той е част от елемента `head`. Указаното заглавие на документ се появява при интерпретация от браузъра в заглавната линия на неговия прозорец. Текстът на заглавието не се включва и не се изобразява в полето на документа. То трябва да съдържа до 40 – 50 знака и да описва най-точно съдържанието на страницата.

Освен заглавието на страницата, между таговете `head` се поставят и специалните метатагове, описващи съдържанието на страницата или указващи някакво поведение на браузъра, търсещите машини, кеширащите системи и други.

- **<body>**

Включва основното съдържание на документа – текст, хипервръзки, изображения, таблици и др. Това е тялото на документа, където се разполага изобразяваната от браузъра информация.

#### Пример:

```
<!DOCTYPE html>
<html>
<head>
<title>Заглавие на документа</title>
</head>
<body>
Съдържание на документа
</body>
</html>
```

**Задача.** Отворете в редактора Notepad++ файла `events.html` от папка "Site" от приложените материали. Като спазвате основните изисквания за структурата на един HTML документ, добавете таговете `<head>`, `<title>`, `<body>`. Заглавието и съдържанието на документа вземете от файла `struktura.docx` от папката "Site\Resources". Запишете файла и проверете какъв е резултатът, като го отворите в браузъра.

### Въпроси и допълнителни задачи

1. Какво представлява тагът?
2. Кои са основните структурни части на HTML документите?
3. Какво се включва в тага `<body>`?

**Форматиране на текст в HTML****Заглавие**

Таговете от `<h1>` до `<h6>` определят заглавия в HTML. `<h1>` е заглавие от най-високо ниво, с най-голям размер на символите. `<h2>` до `<h6>` определят подзаглавията в низходящ ред и с намаляващ размер. Добре е в страницата да има само един `<h1>` елемент и да не се прескачат нивата на подзаглавията, т.е. не трябва да има `<h3>`, ако няма `<h2>`. Браузърите стандартно показват заглавията с удебелен шрифт и празни полета над и под текста на заглавието.

**Параграф**

За обособяване на блок от текст като отделен параграф се използва HTML тагът `<p>`.

**Пример:**

```
<p>Това е параграф.</p>
```

Ако не е зададено друго, браузърите добавят празно поле с размера на един ред преди и след текста, маркиран като параграф.

**Тагове за форматиране на текст:**

`<strong>` – важен текст, показва се удебелен;  
`<em>` – акцентиран текст, показва се курсивен;  
`<b>` – удебелява символите;  
`<i>` – наклонява символите (курсив);  
`<u>` – подчертава символите;  
`<del>` – задрасква символите;  
`<sub>` – създава долен индекс;  
`<sup>` – създава горен индекс.

За структуриране на текстове се използват още:

`<br>` – преминаване на нов ред. Ако направим аналогия с MS Word – `<p>` е равносилно на натискане на "Enter", а `<br>` на "Shift+Enter". Използва се самостоятелно, без затварящ таг. Подходящ е за оформяне на адреси или стихове и когато се налага безусловно преминаване на нов ред. Не е препоръчително да се използва за вмъкване на празно място между параграфи или други елементи.

`<blockquote>` – използва се за вмъкване на цитати. Браузърите вмъкват разстояние колкото един ред преди и след елемента, както и отстъп отляво и отдясно.

`<pre>` – показва текста така, както е написан в текстовия редактор – с всички паузи между букви и думи и оставени празни редове.

**Таг за оформяне на блок.**

Тагът `<div>` `</div>` се използва за оформяне на блок. Той представлява нещо като универсален контейнер, който може да съдържа всякаква информация – текст, изображения, а може и да е празен. С помощта на атрибута `style` (или със CSS, както ще видим по-нататък) може да се задават различни параметри на блока, като размери, цвят, рамка и др. Освен това могат да се задават параметри, които важат за всички разположени в блока обекти. Например за текстове – шрифт, размер, цвят, подравняване и т.н. Това дава възможност за оформяне на структурата на сайта и го прави основен таг за повечето съвременни сайтове.

**Задача 1.** Отворете файла "pois.html" от папката "Site" от приложените материали. В този файл ще добавяме различни елементи в следващите упражнения. Страницата трябва да съдържа списък със задачи за нуждите на уроците. Подбрани са само някои с добавени елементи, но частично оформени. Разгледайте HTML кода на страницата. Ще забележите, че страницата се състои от няколко блокчета, реализирани с тага `<div>`. Номерирали сме ги с помощта на атрибута "id".

**Задача 2.** В момента заглавията на забележителностите са реализирани с таг за параграф. Като използвате тагове за заглавие (от `<h1>` до `<h6>`), оформете заглавието на всяка забележителност (блокче). Кой от таговете ви се струва най-подходящ? Оставете `<h2>` като окончателен вариант.

**Задача 3.** Вземете текста за Исторически музей – Карлово от файла "history\_museum.docx" и като използвате тага за параграф, го поставете в предвиденото за това място (в `div` с `id=2`). Оформете текста само в един параграф. За нови редове използвайте тага `<br>`.

**Създаване на хипервръзка в HTML**

В един HTML документ може да се добавят линкове към други документи или ресурси, които може да се намират на същия сървър или на външен за сайта сървър.

Тагът за създаване на хипервръзка е `<a>`. Използва се най-често с атрибута `href`, който задава адреса на документа (файл или интернет адрес), към който трябва да се свърже.

**Пример:**

```
<a href="https://mon.bg">Връзка към МОН</a>
```

В прозореца на браузъра ще се вижда текстът "Връзка към МОН", при кликане върху който ще се отвори началната страница на сайта mon.bg.

По подразбиране връзките се показват в браузъра сини на цвят и подчертани, а посетените връзки – виолетови. Цветът им може да бъде променен чрез ползване на езика CSS.

Хипервръзките могат да водят към страници от същия сайт, като на `href` се задава относителен път:

```
<a href="events.html">Предстоящи събития</a>
```

или към друг уеб сайт или конкретна страница в него. В този случай на атрибута `href` се задава пълният абсолютен адрес (URL):

```
<a href="https://www.novini.news/events.html">Предстоящи събития</a>
```

Освен към външни страници и към страници от същия сайт, препратките може да водят и към различни части на страницата. Това е подходящо за дълги документи, за да не скролират посетителите в търсене на нужната информация. Използва се атрибутът `id`.

### Пример:

```
<h3 id="glava3">Глава III</h3>
<a href="#glava3">Към 3. глава</a>
<a href="tom1.html#glava3">Към 3. глава на
1. том</a>
```

Първият ред е заглавие на глава от книга със зададен атрибут *id*, което се намира някъде в страницата "tom1.html". Вторият представлява хипервръзка към това заглавие, която се намира в същата страница, напр. в началото ѝ като меню. Третият показва как да използваме същата връзка, но от друга страница, на пример от съдържанието на книгата.

### Пример:

```
<a id="end"></a>
<a href="#end">Към края на книгата</a>
```

Първият ред създава отметка в страницата (невидима), а вторият – и връзка към нея.

Атрибутът *target* определя къде да се отвори свързаният документ, към който сочи хиперлинкът. Приема някои от стойностите: *ime*, *\_blank*, *\_self*, *\_parent*, *\_top*. Най-често се използват:

- \_blank* – отваря документа в нов прозорец;
- \_self* – отваря документа в текущия прозорец (по подразбиране);

- ime* – отваря документа в област с това име (напр. `<iframe id="име">`). Ако няма такава, се отваря в нов прозорец..

**Задача 4.** Отворете файла `pois.html` от папката "Site". Потърсете в него текста "виж на картата" и като използвате тага `<a>` и атрибута `"href"`, направете от текста връзка към сайта `maps.google.com` с реалното местоположение на музея.

- За целта ще ви трябва URL адреса на картата с местоположението на обекта. Влезте в `maps.google.bg` и потърсете музея с подходящи ключови думи.

- След като го намерите, използвайте иконката "Share", за да вземете линк към картата (може да използвате вече готов линк от файла "link.txt" в същата папка).

- Запишете файла и изпробвайте резултата.

- Ако не сте използвали и атрибута *target*, виждате, че картата се отваря на мястото на отворената страница. По-добре е да се отваря в нов прозорец. За целта използвайте атрибута *target* със стойност `"_blank"`.



## Изображения в HTML

Изображенията – картинки, фотографии и др., анимирани или статични, са важна част от почти всеки уеб сайт. В HTML страницата може да се вмъкнат различни формати (gif, jpeg, png) на файлове, представляващи изображения.

Тагът за връзка към изображение е `<img>`. Той се използва без затварящ таг. По подразбиране изображението се появява в лявата част на страницата.

Тагът `img` се използва задължително с атрибутите *src* и *alt*. Атрибутът *src* указва път към изображението или URL адрес. Този път може да е относителен (локален) или абсолютен (пълен).

### Относителен (локален път)

|                                                    |                                                                                   |
|----------------------------------------------------|-----------------------------------------------------------------------------------|
| <code>&lt;img src="picture.jpg"&gt;</code>         | Файлът "picture.jpg" се намира в същата папка като HTML страницата.               |
| <code>&lt;img src="images/picture.jpg"&gt;</code>  | Файлът "picture.jpg" се намира в подпапка "images" на текущата папка.             |
| <code>&lt;img src="/images/picture.jpg"&gt;</code> | Файлът "picture.jpg" се намира в подпапката "images" на основната папка на сайта. |
| <code>&lt;img src="../../picture.jpg"&gt;</code>   | Файлът "picture.jpg" се намира в папката родител на текущата.                     |

### Абсолютен (пълен) път

```

```

Атрибутът *alt* указва алтернативен текст, ако браузърът не зареди изображението. Това, което напишете между кавичките, ще се появи на мястото на изображението ви, ако то не се зареди.

С тага `img` могат да се използват и незадължителни атрибути. Атрибутите *height* и *width* указват съответно височината и ширината на изображението в пиксели (ако те не съвпадат с истинските размери на изображението, тогава изображението ще се "разтегне" или ще се "свие"). Препоръчително е винаги да задавате ширина и височина на изображенията си.

Ако искате да използвате изображението като препратка към друга страница или мултимедия, просто заградете тага `<img>` в тагове `<a>` `</a>` и задайте атрибута *href* на хипервръзката.

**Задача 5.** Отворете файла `pois.html`. Като използвате таг за изображение, добавете подходящо изображение за Исторически музей – Карлово. Файловете се намират в подпапка "history\_museum". Какъв път ще използвате?

- След като добавихте изображението, ще видите, че то е много голямо. За да зададем с какъв размер да се показва на страницата, трябва да зададем стойност на атрибутите *height* и *width* в тага `<img>`. Най-добре е да се задават и двата атрибута, но съществува опасност, ако не са спазени пропорциите на зададените и истинските размери, изображението да е сплескано. Затова задайте само `height="300"`, запишете файла и проверете резултата в браузъра. Вместо *height* задайте *width* и пак проверете резултата.

### Важно

- Препоръчително е таговете и атрибутите да се пишат с малки букви, въпреки че HTML не прави разлика.
- Стойностите на атрибутите се заграждат в кавички (" ").
- Може предварително да се извърши форматиране на информацията за страницата в програмата MS Word и тогава да се започне писането на HTML страница.

**Тагове за таблица**

Тагове за таблица:

- `<table>` – създава таблица
- `<tr>` – създава ред в таблица
- `<td>` – създава клетка в ред от таблица
- `<caption>` – заглавие на таблица
- `<th>` – заглавие на колона в таблица

Клетките в таблицата имат атрибути:

*colspan* – определя броя на колоните, които клетката трябва да обхване

*rowspan* – определя броя на редовете, които клетката трябва да обхване

Вместо следващите атрибути в HTML5 се използва CSS:

*border* – определя рамката на таблицата

*cellspacing* – определя разстоянието между клетките

*cellpadding* – определя отстъпа на текста в клетката

*height* – определя височината на таблицата, реда или колоната

*width* – определя дължината на таблицата или колоната

*align* – подравнява таблица, ред или клетка

*bgcolor* – задава фонов цвят на таблица, ред или клетка

**Пример:**

```
<table border="1">
<tr>
<th>Колона 1</th>
<th>Колона 2</th>
<th>Колона 3</th>
</tr>
<tr>
<td>ред 1, колона 1</td>
<td>ред 1, колона 2</td>
<td>ред 1, колона 3</td>
</tr>
<tr>
<td>ред 2, колона 1</td>
<td>ред 2, колона 2</td>
<td>ред 2, колона 3</td>
</tr>
</table>
```

| Колона 1        | Колона 2        | Колона 3        |
|-----------------|-----------------|-----------------|
| ред 1, колона 1 | ред 1, колона 2 | ред 1, колона 3 |
| ред 2, колона 1 | ред 2, колона 2 | ред 2, колона 3 |

**Задача 1.** Отворете файла `events.html`. В него създайте таблица в подходящия блок (`div`), като използвате данните от файла `"Site\Resources\Karlovovo_tablitsa.docx"`. За целта може да създадете първо един празен ред (`<tr>`) с 2 колони (`<td>`), като след това го размножите, за да получите нужните редове. Запишете файла и го проверете.

**Тагове за списъци**

Списъците могат да бъдат с номерация (подредени) и със символи (неподредени). Подредените списъци започват с тага `<ol>`, а всеки елемент на списъка – с тага `<li>`:

**Пример:**

```
<ol>
<li>Едно</li>
<li>Две</li>
</ol>
```

1. Едно

2. Две

Атрибутът *start* задава началото на номерацията, а присвоената му стойност показва началото на номерацията.

```
<ol start="3">
<li>Едно</li>
<li>Две</li>
<li>Три</li>
</ol>
```

3. Едно

4. Две

5. Три

Атрибутът *reversed* указва, че подреждането ще е в низходящ ред, а присвоената му стойност показва началото на номерацията:

```
<ol reversed start="4">
<li>Едно</li>
<li>Две</li>
<li>Три</li>
</ol>
```

4. Едно

3. Две

2. Три

Атрибутът *type* задава вида на номерацията (букви или цифри), например 1, а, А, i, I:

```
<ol type="I">
<li>Едно</li>
<li>Две</li>
<li>Три</li>
</ol>
```

- I. Едно
- II. Две
- III. Три

Неподредените списъци започват с тага `<ul>`, а всеки елемент на списъка – с тага `<li>`:

```
<ul>
<li> Едно </li>
<li> Две </li>
</ul>
```

- Едно
- Две

**Задача 2.** Отворете файла `pois.html` от папката "Site". Преобразувайте последните два параграфа от текста за първия обект (`<div id="1">`) в неподреден списък. Запишете файла и го проверете.

### Таг за аудио

Таг за аудио – `<audio>`. Използва се за вграждане на звукови файлове. Включва възпроизвеждане на звук, синтез и запис, също така описва техники за разпознаване на реч. Той осигурява контроли (възпроизвеждане, пауза, сила на звука), които може да изглеждат различно в различните браузъри и операционни системи.

Основните атрибути, използвани с тага **audio**:

*src* – път (URL) към файла

*preload* – предварително зареждане на файл

*autoplay* – автоматично възпроизвеждане на файл

*loop* – циклично възпроизвеждане

*muted* – без звук по подразбиране при стартирането

*controls* – контроли за управление на звук

Тагът **audio** може да съдържа един или повече аудиоизточници, представени с помощта на атрибута *src* или тага `<source>`, като браузърът избира подходящия.

#### Пример:

```
<audio controls>
<source src="music.mp3"/>
<source src="music.ogg"/>
<source src="music.flac"/>
</audio>
```

### Таг за видео

Таг за видео – `<video>`. Използва се за вграждане на видеофайлове.

Източникът на видеофайла може да бъде зададен по два начина. По-краткият е даден в пример 1.

Различните браузъри поддържат много на брой видеоформати. Чрез използване на тага

`<source>` (пример 2) има възможност за изброяване на файлове с различни медийни формати, като браузърът автоматично пробва подред да зареди всеки файл. Както е съставен пример 2 – ако браузърът не успее да възпроизведе никой от трите видеофайла, ще изпише съобщението под тях.

Основните атрибути, използвани с тага **video**:

*src* – път (URL) към файла

*preload* – предварително зареждане на файл

*autoplay* – автоматично възпроизвеждане на файл

*controls* – контроли за управление на възпроизвеждането

*poster* – показва изображение на мястото на видеоклипа, преди да започне възпроизвеждането

*type* – указва типа на видеото (MIME). Може да включва и списък с кодеци, които помагат на браузъра да определи дали може да декодира файла.

*loop* – циклично възпроизвеждане

*autoplay* – автоматично стартиране на видеото със зареждане на страницата

*muted* – без звук по подразбиране при стартирането

*height, width* – размери на видеоелемента (видеооплейъра)

#### Пример 1:

```
<video controls width="640" src="movie.mp4" type="video/mp4">
</video>
```

#### Пример 2:

```
<video poster="movie.jpg" controls
height="640" width="480">
<source src="movie.mp4"
type="video/mp4"/>
<source src="movie.webm"
type="video/webm"/>
<source src="movie.ogv"
type="video/ogg"/>
Your browser does not support this
video type!
</video>
```

**Задача 3.** Отворете файла `index.html` от папката "Site". Потърсете таг `<div id="video">` и вътре добавете видеофайла `"rosa.mp4"` от подпапката `"video"`.

- Използвайте атрибута **poster** за указване на начално изображение на видеоелемента, като използвате файла `"poster.png"` от папката на видеото.

- Изпробвайте различни стойности за атрибута **width**, например `"600"`. Какви пътища ще зададете? Запишете файла и го изпробвайте.

Формите (формуляри) представляват част от документ, позволяваща на потребителя да въвежда информация за последваща обработка от системата. Във формите се използват различни елементи (контроли, етикети и др.), в които може да се избере или въведе определена информация, да се отбележат предложените възможности.

Всяка форма започва с отварящ таг `<form>` и завършва със затварящ таг `</form>`. Цялото съдържание на формата е заключено между тези два тага.

Повечето елементи на формите се въвеждат чрез тага `<input>`, който може да съдържа няколко атрибута, най-важният от които е `type`. Атрибутът `type` указва вида на елемента в конкретния случай.

## Пример:

```
<form>
<input type="text" />
<input type="hidden" />
<input type="password" />
</form>
```

## Поле за текст

Почти всяка форма съдържа поле за въвеждане на текст. На страницата тези полета се показват като правоъгълници, в които може да се въвежда текст. За да се зададе поле за въвеждане на текст във форма, трябва да се въведе атрибутът `type` на тага `input` и да му се зададе стойност `text`:

## Пример:

```
<input type="text" />
```

В този случай на страницата ще се изобрази текстово поле, което по подразбиране има дължина 20 символа. Препоръчително е освен атрибута `type`, в тага `input` да се използват и следните атрибути:

- Атрибута `name`, който приема като стойност името за полето.
- Атрибутът `size` задава дължината на текстовото поле в брой символи (например `size="40"`). Ако не се използва този атрибут, дължината на текстовото поле по подразбиране ще бъде 20 символа.
- Чрез атрибута `value` може да се зададе текст, който да се вижда в текстовото поле, преди още клиентът да е въвел някаква информация.
- Чрез атрибута `maxlength` може да въведете ограничение на броя на изписваните в текстовото поле символи. Например ако зададете `maxlength="15"`, в текстовото поле няма да бъде възможно да се въведат повече от 15 символа (букви, цифри и други).

За да въведете текст, който да обяснява на потребителя предназначението на съответното поле, използвайте елемента `<label>`. Той създава етикет, свързан с полето чрез неговото `id`, зададено в атрибута `for`.

## Пример:

```
<form>
<label for="Name">Вашето име:</label>
<input type="text" name="Name" id="Name"
  size="30" maxlength="35" />
</form>
```

Примерът създава форма с 1 текстово поле с етикет "Вашето име". То има дължина 30 знака и ограничение за въвеждане на повече от 35 символа.

## Поле за отметка

Полетата за отметки се използват във формите, за да може посетителят на страницата да избере едновременно няколко възможности. Създават се няколко елемента `<input>`, на които се задава атрибут `type` със стойност `checkbox`. На всеки се присвоява двойка атрибути `name` и `value`.

Полетата, които потребителят отметне, ще изпратят към сървъра съответната комбинация `name=value`.

Използваме елемента `<label>` за описание на полето.

## Пример:

```
<form>
<input type="checkbox" id="device1"
  name="device1" value="Phone">
<label for="device1"> Телефон</label><br>
<input type="checkbox" id="device2"
  name="device2" value="Tablet">
<label for="device2"> Таблет</label><br>
<input type="checkbox" id="device3"
  name="device3" value="Laptop">
<label for="device3"> Лаптоп</label>
</form>
```

В примера създаваме форма за въпроса "Какви IT устройства използвате?" с три отговора, които могат да бъдат избрани едновременно. Тя ще изпрати от 0 до 3 стойности във формат `name=value`, напр.

`device1=Phone&device3=Laptop`

## Радиобутон

Радиобутоните се използват за избор между няколко възможности, като позволяват да се избере само една. За да се изработи радиобутон, на атрибута `type` от елемента `<input>` се задава стойност `radio`. На атрибута `name` на всички елементи от групата радиобутони се задава една и съща стойност. Останалите атрибути са като при полетата за маркиране. Радиобутоните изпращат само една двойка `name=value`.

## Пример:

```
<form>
<input type="radio" id="device1"
  name="device" value="Phone">
<label for="device1"> Телефон</label><br>
<input type="radio" id="device2"
  name="device" value="Tablet">
<label for="device2"> Таблет</label><br>
<input type="radio" id="device3"
  name="device" value="Laptop">
<label for="device3"> Лаптоп</label>
</form>
```

Този път създаваме форма за въпроса "Какъв подарък искате?" с три отговора, като можете да изберете само един от тях. Тя ще изпрати само една стойност във формат `name=value`, напр. `device=Phone`. Забележете, че атрибутите `id` винаги са уникални. В случая се използват за връзка с `<label>`.

## Важно

При използване на елементите `form`, `select`, `option`, `textarea` изпускането на затварящия таг ще доведе до сериозни обърквания и грешки.

## Падащо меню

Елементът `<select>` се използва за създаване на списък, от който потребителят избира една или няколко стойности (падащо меню). Задължителен атрибут е *name*. Той определя под какво име избраните стойности ще се изпратят към сървъра.

Тагът `<select>` загражда нужния брой елементи `<option>`, които съставят списъка. Техните атрибути *value* задават стойностите, които ще бъдат върнати, а текстът, затворен между таговете `<option>` и `</option>`, ще се визуализира в падащото меню

Пример:

```
<form>
Кой е любимият ви цвят? <br />
<select name="ChooseOne">
<option value="Red">червен</option>
<option value="Blue">син</option>
<option value="Green">зелен</option>
</select>
</form>
```

Падащите менюта дават възможност както за единичен, така и за множествен избор. За да бъде възможно да се маркира повече от едно поле от падащото меню, в тага `select` е нужно да се въведе атрибутът *multiple*. Атрибутът *size* задава броя на видимите опции. По подразбиране е 1. При по-голяма стойност менюто ще се покаже като списък с лента за превъртане.

## Текстова област

Ако желаете да предоставите възможност за по-подробно мнение или коментар, трябва да използвате многоредово поле за въвеждане на текст. Многоредовите полета се въвеждат чрез тага `<textarea>`. Атрибутите на тага са:

- Атрибутът *name*, определящ названието, под което ще се съхранява и обработва тази част от формата.
- Атрибутът *rows*, на който се присвоява като стойност някаква цифра, указваща височина на полето в брой редове.
- Атрибутът *cols*, на който се присвоява като стойност някаква цифра, указваща ширината на полето в брой символи (цифри, букви и др.).

Ако искате да зададете текст, който да се покаже в полето, трябва да го изпишете между отварящия и затварящия таг.

Ако желаете да поясните предназначението на многоредовото поле, изпишете текста преди тага `<textarea>`, най-добре в елемент `<label>`.

Пример:

```
<form>
<label for="text1">Вашият коментар:</label>
<textarea name="Comentar" id="text1"
cols="40" rows="12">Моят коментар е
</textarea>
</form>.
```

Това е пример за форма с текстова област с име `Comentar`, с дължина 40 символа и височина 12 реда. Указващият текст към полето да бъде "Вашият коментар:", а вътре в самото поле да се вижда текстът "Моят коментар е".

Ако коментарът е дълъг и не се събере във видимата част на полето, отстрани ще се появи лента за превъртане.

## Поле за въвеждане на парола

За да изработите поле за въвеждане на парола, трябва да зададете на атрибута *type* от тага `input` стойността `password`.

Пример:

```
<form>
<label for="pass">Въведете парола:</label>
<input type="password" id="pass"
name="parola" />
</form>
```

## Поле за качване на файлове

За да изработите поле за качване на файлове, трябва да зададете на атрибута *type* от тага `input` стойността `file`. С атрибута *name* се задава име, под което данните ще се изпратят към сървъра. Може да се използва и атрибутът *multiple* за качване на няколко файла едновременно. След изписване на кода в прозореца на браузъра се изобразява поле и бутон с надпис **Browse**. При натискане върху бутона се появява прозорец с файловете от компютъра, от който той избира нужния за прехвърляне файл.

Пример:

```
<form>
<input type="file" name="UploadFile"
size="25" />
</form>
```

## Бутон за изпращане (изтриване) на данни

За изпращане и обработка на попълнените данни в дадена форма се използва бутонът "Submit". Обикновено този бутон може да се види в края на формата. За да изработите бутон за изпращане на данни, трябва да зададете на атрибута *type* от тага `input` стойността `submit`.

Пример:

```
<form>
<input type="submit" value="Submit" />
</form>
```

Аналогично е изработването на бутон "Reset", чрез който формата се изчиства и се изтриват попълнените вече данни. За целта трябва да зададете на атрибута *type* от тага `input` стойността `reset`.

Други атрибути на `<form>`:

*method* – определя метода за изпращане на формата

*action* – задава URL адрес, на който да се изпрати, по подразбиране е адресът на формата

*name* – определя името на формата

Други атрибути на `<input>`:

*type=image* – определя външно изображение като бутон за изпращане

*type=hidden* – определя скрито поле

*value* – задава начална стойност на полето

*name* – определя името на полето при изпращане на данните

Формата се изпраща на URL адреса, посочен в атрибута *action* по два метода: **GET** и **POST**. Първият добавя данните от полетата към URL адреса като списък формат *име=стойност*, разделени със знака `&`. Това е лесно за проследяване и запомняне като отметка в браузъра, но не е подходящо за чувствителна информация, а адресът не може да е по-дълъг от 3000 символа. Методът **POST** вмъква данните в тялото на HTML заявката. Така те не се виждат в полето за адрес на браузъра и отпада ограничението за обема им.

**Задача.** Отворете редактора BlueGriffon и създайте файл с име `form.html`, който да представлява регистрационна форма със следните данни: Име (текстово поле), Години (падащо меню), Пол (радиобутон), Потребител (текстово поле), Парола (поле за парола), Имейл (текстово поле), Бутон за изпращане (Submit).

### Нива на CSS

Характерно за развитието на CSS е, че промените и нововъведенията не отменят текущия стандарт, а само го допълват и разширяват. Затова езикът има нива, а не версии. W3C приема първото ниво – CSS1, в края на 1996 г. CSS2.1 (2011 г.) е текущият пълен стандарт. Ниво CSS3 е в етап на активна разработка. При него спецификациите са разделени в отделни модули, които се развиват независимо. Много от тях вече са приети от W3C, другите са на различни фази на разработка.

### Мерни единици в CSS

Стойностите на размерите се задават с числа и мерни единици. Те са абсолютни – **px** (пиксел), **mm**, **cm**, и относителни – **em** (част от размера на шрифта на елемента, напр. ако шрифтът е 10px, 0.5em или 2em, ще означава съответно 5px и 20px), **rem** (като **em**, но отнесено към шрифта на коренния елемент на страницата), **%** (процент от размера на родителския елемент) и др. Мерната единица се пише **слято** с числото!

### Стил (Style)

Атрибутът **style** се използва за задаване на визуален стил на даден HTML елемент, например цвят, размер, шрифт и др. Форматът му е:

```
<tagname style="property: value;">
```

#### Пример:

```
<h1 style="color: blue;">Заглавие в синьо</h1>
```

Задава цвета на текста в елемента **h1** като син.

```
<p style="background-color: #ffff00;">Това е параграф с  
жълт фон.</p>
```

Задава цвета на фона на елемента **p** като жълт.

Този стил се нарича **вграден (inline)** и важи само за елемента, за който е зададен.

**Задача 1.** Задайте **оранжев фон** на заглавието на първия обект (**div id="1"**) в страницата **"site\pois.html"** от приложените материали.

За целта трябва да отворите страницата и да намерите тага **<h2>**, с който е оградено заглавието. Променете го, като му добавите атрибута **style="background-color: #ff6600;"** (**#ff6600** е код на **оранжев цвят**). Запишете файла и го тествайте в браузъра.

По този начин променихме цвета на един елемент. За да направим това за всички подобни елементи, можем да използваме т.нар. **CSS**.

### Cascading Style Sheets (CSS)

**CSS** (англ. Cascading Style Sheets – каскадни таблици със стилове) е език за описание на стилове, който се използва заедно с HTML. Той е създаден за лесно управление на визуалното представяне на съдържанието (шрифтове, размери, цветове, фонове, полета и др.), без да се налага промяна на самите HTML елементи. CSS е мощен инструмент, който с редактиране на един или няколко параметъра може да промени външния вид на целия сайт.

CSS може да се приложи по 3 начина:

- **Вграден (inline)** – чрез атрибута **style** на конкретен елемент, важи само за него (както направихме в предната задача).

- **Вътрешен (internal)** – като блок със стилове, затворени в тага **<style>**, засяга елементите само в текущата страница.

- **Външен (external)** – стиловете са описани в отделен текстов файл с разширение **.css**. Връзката към него се задава с елемент **<link>** в заглавната част **<head>**. Това дава възможност да се приложи един набор от стилове към десетки уеб страници в голям сайт, както и да се задават комбинации от няколко **.css** файла в зависимост от съдържанието на страницата.

#### Пример:

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" type="text/css" href="styles.css">
</head>
<body>
  .....
</body>
</html>
```

**Задача 2.** Отворете файла **index.html** от папката **"site"**. Намерете къде има **вмъкната връзка към css файл**. Вижте името му, отворете го в редактора **Notepad++** и го разгледайте.

## Работа с езика Cascading Style Sheets (CSS)

CSS кодът представлява последователност от правила, всяко от които започва със селектори, последвани от блок с дефиниции от свойства и стойности, които им се присвояват. Обобщената структура на едно CSS правило е:

```
селектор1, селектор2, ... {  
    свойство1: стойност;  
    свойство2: стойност;  
    .....  
}
```

Правилата са каскадни, т.е. приложени към даден елемент, те се разпростират върху всичките му подчинени елементи. Например правило за размер на текст, приложено към таблица, ще важи за текста във всичките ѝ клетки, а приложено към елемента `<body>` – за всички елементи на страницата, които съдържат текст.

Двойките свойство – стойност се разделят с двоеточие (:), а всяка такава дефиниция завършва с точка и запетая (;). Списъкът от всички свойства се загражда в големи скоби {...}. Тъй като новите редове се игнорират, цялото правило и дори целият CSS файл могат да се напишат на един ред или да се разделят за по-добра четивност.

**Селекторът** е шаблон, описващ по определени правила елементите, върху които се прилага правилото. Например селекторът "div" ще съвпадне с всички `<div>` елементи, а селекторът "div h1" – с елементите `<h1>`, които са вложени в `<div>` елемент. Селекторите могат да бъдат няколко, разделени със запетая. В такъв случай зададените свойства се отнасят за всеки от тях.

**Задача 3.** Сменете цвета на всички заглавия на забележителности в страницата `pois.html`, като използвате наличния CSS файл от папката "site".

Можем да използваме факта, че всяка забележителност е сложена в таг `<div>` с `class="poiBlock"`. Затова чрез селектора ".poiBlock h2" можем да влияем само на елемента `h2` в `<div class="poiBlock">`, без да засегнем останалата част от страницата. Отворете в Notepad++ CSS файла от разработвания сайт и намерете горния селектор. Той е създаден, но няма стойности. Добавете в него реда "background-color: #f3a032;". В него "background-color" е свойство, а "#f3a032;" – стойност. Запишете CSS файла и проверете как се е променила страницата `pois.html`.

Когато задаваме стойности в CSS, не трябва да има интервал между числото и мерната единица, напр. "10px" или "80%". На някои свойства може да се задават няколко стойности наведнъж, като те се разделят с интервал, напр. "border: solid 1px red;" ще зададе на елемента червена, плътна рамка с дебелина 1 пиксел. Може да се зададе списък от стойности, разделени със запетая, напр. "font-family: Tahoma, sans-serif" ще форматира текста с шрифт Tahoma или, ако не е наличен – с шрифт sans-serif. Ако стойността съдържа интервал, напр. ако името на шрифта е Open Sans, то се поставя в кавички (единични или двойни).

**Пример:**

```
p, div {  
    font-family: Calibri, 'Open Sans', sans-serif;  
    font-size: 10px;  
    border: solid 1px red;  
    line-height: 130%;  
}
```

Това правило ще се приложи към всички `<p>` и `<div>` елементи. То ще направи шрифта на символите Calibri с размер 10 пиксела, височина на реда – 130% от размера на шрифта, т.е. 13 пиксела, и ще ограда елементите с червена рамка. Ако не намери шрифта Calibri, браузърът ще потърси Open Sans, а при неуспех – sans-serif.

**Задача 4.** Отворете файла "styles.css" и променете правилото за `<body>`, като напишете интервал в размера на шрифта (font-size: 12 px;). Вижте как се промени страницата.

## Цветове в CSS

В CSS се използват няколко формата за описание на цветове. Най-често използвани са:

- **с име** – в HTML и CSS са специфицирани 140 имена на цветове на английски, напр. red, green, coral, gold, indigo и т.н.

- **#RRGGBB** – шестнайсетичен код, където RR е шестнайсетичната стойност на червеното в желания цвят, описан по RGB цвето-ва схема. Може да приема стойности от 00 до ff, което съответства на 256 степени (нюанса). GG и BB са съответно стойностите на зелено и синьо. Така черен цвят се описва като #000000, бял – #ffffff, червен – #ff0000, жълт – #ffff00, и т.н. Някои цветове могат да се запишат кратко с 3 цифри (#RGB), напр. #f00 е същото като #ff0000 (червен), а #ff0 е жълт (#ffff00).

- **rgb(red, green, blue)** – функция с параметри интензитетите на трите основни цвята (червен, зелен, син), зададени като степен от 0 до 255 или като процент. Червеният цвят ще се опише като rgb(255, 0, 0) или rgb(100%, 0%, 0%).

- **rgba(red, green, blue, alpha)** – функция, подобна на rgb(), но с допълнителен параметър за прозрачност, който приема стойности от 0.0 (напълно прозрачен) до 1.0 (съвсем плътен). Напр. rgba(255, 0, 0, 0.5) е червено с 50% прозрачност. Същото може да се зададе и като #ff000080.

## Коментар в CSS

Коментарът е пояснителен текст, който се игнорира от браузъра. Форматът му в CSS е:

```
/* Това е коментар */
```

## Основни понятия

**CSS** (англ. Cascading Style Sheets)  
– каскадни таблици със стилове)  
– език за описание на стилове.

Морето е синьо

Морето е синьо

Морето е синьо

Фиг. 1. На първия ред се вижда параграфът без приложени CSS правила.

На втория е резултатът от прилагането на правилото от задача 5.

На третия – резултатът от задача 6.

## Видове селектори в Cascading Style Sheets (CSS)

### Селектори за клас

В CSS можем да групираме множество от елементите посредством техния атрибут клас (**class**). При използване на класа като селектор правилото ще се приложи към всеки елемент, на който е присвоен този клас. Можем да използваме следния синтаксис:

**елемент.клас {свойство: стойност;},**

за да приложим правилото към всички елементи, които са едновременно от зададения вид и клас.

#### Пример:

```
h1.red {color: #ff0000;}  
h2.red {color: #ff0000;}
```

Тези правила ще направят червен текста на заглавията от вид `<h1>` и `<h2>`, на които в HTML кода сме задали атрибут `class="red"`.

Класовете могат да се използват като селектор и без да се обвързват с конкретен вид елементи. В този случай синтаксисът на CSS кода е:

**.клас {свойство: стойност;}**

#### Пример:

Можем да оптимизираме предишния пример така:

```
.red {color: #ff0000;}
```

Този селектор ще приложи правилото към всички елементи с клас "red", без значение от кой вид са – `<h1>`, `<h2>` или произволен друг.

**Задача 5.** Създайте правило за клас с име "right" със следните характеристики: позициониране на текста вляво, шрифт Calibri, размер на символите 10 пиксела, син цвят на символите.

```
.right {  
  text-align: right;  
  font-family: Calibri;  
  font-size: 10px;  
  color: blue;  
}
```

Така дефинираното правило за класа се използва в HTML документа, като се задава атрибутът *клас* на избраните елементи (фиг. 1):

```
<body>  
  <p class="right">Морето е синьо</p>  
</body>
```

### Групиране на селектори

В едно правило могат да се използват няколко селектора, като се отделят със запетаи. Синтаксисът при групирането е:

**селектор1, селектор2, селектор3, ... {свойство: стойност;}**

#### Пример:

```
h1, h2, h3, p {color: #ff0000;}
```

В примера са групирани селекторите за първите 3 по големина заглавия и за параграф.

### ID селектори

С помощта на ID селектори могат да се декларират параметри, които са специфични за конкретен елемент, на който в HTML кода сме задали атрибут "id". Тъй като този атрибут трябва да е с уникална стойност за всеки елемент, правилото ще се приложи само за него и елементите, на които той е "родител". За тази цел се използва следният синтаксис:

**#id {свойство: стойност;}**

**Задача 6.** Добавете червен фонов цвят на създадения в предната задача параграф, като използвате ID селектор с име *red*.

Към CSS правилото със селектор ".right" от задача 5. добавяме:

```
#red {background-color: red}
```

В HTML страницата ще въведем следния код (фиг. 1):

```
<body>  
  <p id="red" class="right">Морето е синьо</p>  
</body>
```

Селекторите за вид, клас и ID на HTML елементите могат да се ползват както самостоятелно, така и в комбинации помежду си.

## Основни свойства

Основните свойства, които се използват при работата с езика CSS, са сходни с част от атрибутите на таговете в езика HTML.

Ето накратко някои основни, често използвани свойства:

- **width** – задава дължината на елемента;
- **height** – задава височината на елемента;
- **opacity** – задава прозрачност на елемента в интервала 0 – 1 или 0% – 100%;
- **padding** – задава разстояние между съдържанието на елемента и неговата рамка. Приема до четири стойности, разделени с интервали, съответно за горната, дясната, долната и лявата страна. Вижте "Задаване на списък от стойности" по-долу;
- **margin** – задава разстояние между рамката и околните елементи. Форматът на стойностите е като на padding;
- **display** – определя видимостта на елемента. Възможните стойности са: none (скрива елемента), inline (показва елементите един след друг и игнорира стойностите за размер), block (елементът се изобразява като блок, започва на нов ред и заема цялата ширина);

• **position** – задава начина на позициониране: static (елементите се показват по реда на срещане в HTML кода), absolute (абсолютно – спрямо най-близкия нестатичен родителски елемент), relative (относително – спрямо нормалното си разположение), fixed (фиксирано – спрямо прозореца на браузъра);

• **left, right, top, bottom** – задават отместване на елемента съответно отляво, отдясно, отгоре, отдолу при позициониране, различно от статично;

• **float** – "плаване". Позволява даден елемент да се обтича от текст и други inline обекти. Игнорира се за елементите с абсолютна позиция. Възможните стойности са: none (не "плава", стойност по подразбиране), left (позиционира се най-вляво на родителския елемент, като позволява текстът и другите елементи да го обтекат отдясно), right (като left, но вдясно);

• **clear** – определя от коя страна на елемента не може да има плаващи елементи. Стойности: left (отляво), right (отдясно), both (от двете страни);

• **z-index** – задава реда на елементите при позициониране – тези с по-малък индекс са под тези с по-голям индекс.

### Задаване на списък от стойности

Някои свойства, напр. margin и padding, които се отнасят до четирите страни на елемента, могат да приемат списък от 1 до 4 стойности, разделени с интервал:

• 4 стойности (напр. margin: 1px 2px 3px 4px;) – отнасят се съответно за горната (1px), дясната (2px), долната (3px) и лявата (4px) страна;

• 3 стойности (напр. margin: 1px 2px 3px;) – отнасят се за горната (1px), дясната и лявата (2px) и долната (3px) страна;

• 2 стойности (напр. margin: 1px 2px;) – отнасят се съответно за горната и долната (1px) и дясната и лявата (2px) страна;

• 1 стойност (напр. margin: 1px;) – отнася се за четирите страни.

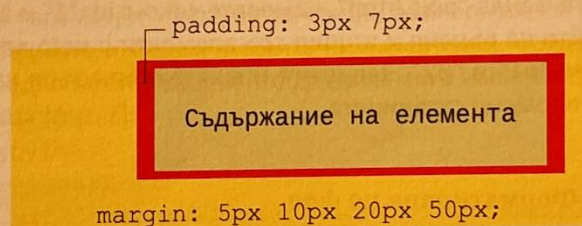
Тези параметри могат да се задават и като отделни свойства с по една стойност (напр. margin-left: 2px; или padding-top: 5px; ).

### Пример:

```
div {
    margin-top: 5px;
    margin-right: 10px;
    margin-bottom: 20px;
    margin-left: 50px;
    padding-top: 3px;
    padding-right: 7px;
    padding-bottom: 3px;
    padding-left: 7px;
}
```

Това правило е аналогично на:

```
div {
    margin: 5px 10px 20px 50px;
    padding: 3px 7px;
}
```



**Задача 1.** Стартирайте "pois.html" в браузъра. Отворете файла "styles.css", намерете правилото със селектор ".poiBlock img" и променете стойността на "float" първо на "left", а след това на "none". Определете страницата в браузъра и опишете как се отразяват промените. Върнете я пак в начално състояние.

## Форматиране на текст

Основни свойства за форматиране на текст:

• **color** – цвят на символите в CSS формат.

• **text-align** – подравняване на текст с възможни стойности: left (ляво), right (дясно), center (центрирано), justify (двустранно).

• **font-size** – размер на символите.

• **font-family** – шрифт на текст.

• **font-weight** – задава "тегло" на текста. Може да приема стойности: **normal** (нормален текст), **bold** (удебелен текст), **lighter** (изтънен текст) и др.

• **text-decoration** – задава допълнителен ефект на текста. Възможните му стойности са: **underline** (подчертава текста с долна черта), **none** (без ефекти), **overline** (подчертава текста с горна черта), **blink** (текстът премигва), **line-through** (зачертава текста).

• **text-indent** – задава отстъп на първия ред на абзац.

• **letter-spacing** – променя разредката между символите. Може да бъде normal (без промяна на разредката) или число с мерна единица. Може да приема отрицателни стойности.

• **word-spacing** – променя разстоянието между думите аналогично на **letter-spacing**.

• **line-height** – задава височината на реда (междуредието).

• **text-transform** – управлява главни и малки букви: **none** (без промяна, по подразбиране), **uppercase** (само с главни букви), **lowercase** (само с малки букви), **capitalize** (всяка дума започва с главна буква).

**Пример:**

```
p {
  text-align: justify;
  font-family: 'Times New Roman';
  font-size: 12px;
  color: #00ffff;
  font-weight: bold;
  line-height: 1;
}
```

**Задача 2.** Отворете файла "styles.css", намерете правилото със селектор ".poiText p" и го копирайте два пъти, като промените селекторите на "p.p1" и "p.p2". В първото от новите правила добавете цвят на текста и го направете удебелен. Във второто задайте друг цвят с функцията **rgb()** и променете "margin" така, че отляво да е 10 пиксела. Запишете. Отворете файла "pois.html", намерете `<div>` с `id="1"` и задайте на първия и втория `<p>` елементи в него класове "p1" и "p2". Запишете и вижте в браузъра как се промени страницата.

## Форматиране на фон

Външният вид на уеб сайта зависи от използването на подходящи фонове както на целите страници, така и на отделните елементи.

Цветът на фона се задава, като на свойството **background-color** се присвоява стойност в някой от приложимите формати.

**Пример:**

```
h1 {background-color: red;}
```

За фон на уеб елемент може да се използва и изображение. То се задава с **background-image**. За посочване на пътя до избраното изображение се използва функцията **url('URL')**.

**Пример:**

```
body {
  background-image: url('proba.gif');
}
```

По подразбиране изображението се повтаря вертикално и хоризонтално до запълване на целия елемент. Свойството **background-repeat** служи за управление на запълването.

Приема стойности: **repeat** – изображението се повтаря както хоризонтално, така и вертикално, **repeat-x** (**repeat-y**) – изображението се повтаря само хоризонтално (вертикално), **no-repeat** – изображението не се повтаря.

**Пример:**

```
body {
  background-image: url('proba.gif');
  background-repeat: repeat-y;
}
```

В примера изображението ще се повтаря по вертикала.

Свойството **background-attachment** определя дали фоновото изображение ще е фиксирано спрямо видимата област от страницата (**fixed**), или ще се движи при превъртане с елемента (**scroll**).

**Пример:**

```
body {
  background-image: url('proba.gif');
  background-repeat: repeat-y;
  background-attachment: fixed;
}
```

В примера изображението е фиксирано.

Свойството **background-position** определя началната позиция на изображението. Задават се хоризонтална и вертикална стойност в проценти или мерни единици, или комбинация от служебните думи **left**, **center**, **right** (за хоризонтално позициониране) и **top**, **center**, **bottom** (за вертикално). Ако е посочена само една стойност, другата се приема за **center**.

**Пример:**

```
body {
  background-image: url('banner.jpeg');
  background-repeat: no-repeat;
}
body { background-position: right top; }
/* аналогично на background-position:
   100% 0% */
body { background-position: top center; }
/* 50% 0% */
body { background-position: center; }
/* 50% 50% */
body { background-position: bottom; }
/* 50% 100% */
```

**Задача 3.** Отворете файла "styles.css" и намерете правилото със селектор "#pois". Обяснете какво е действието на всяка дефиниция в него. Редактирайте ги по ваш избор и вижте в браузъра какъв ще е ефектът върху страницата "pois.html".

## Използване на рамки

Основни свойства за рамки:

• **border-color** – цвят на рамката.

• **border-style** – стил на рамката, който има няколко варианта (**none**, **dotted**, **dashed**, **solid**, **double**, **groove**, **ridge**, **inset**, **outset**), фиг. 1.

• **border-width** – дебелина на рамката, която се определя в пиксели или друга мерна единица.

Изброените свойства могат да се задават като списък от до 4 стойности (както описахме по-горе) или поотделно, напр. **border-top-color**, **border-right-width**, **border-bottom-style** и т.н. Много често рамката се задава в съкратен формат:

**border: border-width border-style border-color;**

Пример:

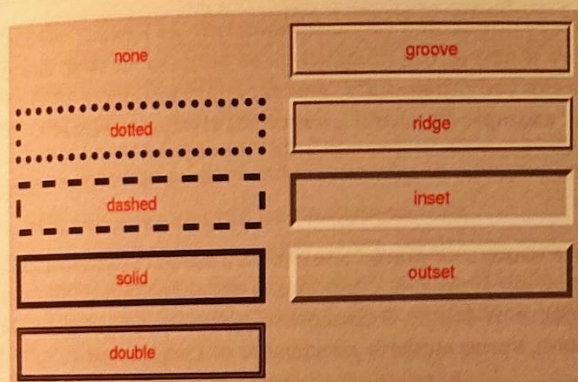
```
div { border: 1px solid red; }
```

• **border-radius** – определя радиуса на закръгляне на върховете на елемента. Както и при останалите свойства на рамката, могат да се задават до 4 стойности, като тук ще се отнасят съответно за горен ляв, горен десен, долен десен и долен ляв ъгъл.

Пример:

```
div {
  width: 100px;
  height: 70px;
  border-color: black red;
  border-style: dashed solid;
  border-width: 5px 10px;
  border-radius: 10px 20px 15px;
}
```

Правилото ще направи горната и долната рамка черни, пунктирани, дебели 5 пиксела, а дясната и лявата – червени линии с дебелина 10 пиксела. Ъглите ще са закръглени съответно горен ляв – 10, горен десен – 20, долен десен – 15, и долен ляв – 20 пиксела.



Фиг. 1

**Задача 4.** Отворете файла "styles.css" и намерете правилото със селектор ".poiBlock". Направете рамката червена с дебелина 5 пиксела. Задайте закръгляне на горния ляв ъгъл с радиус, 3 пъти по-голям от размера на шрифта. Използвайте свойството **border-top-left-radius** и мерната единица "em".

### Добавяне и използване на шрифтове

Както споменахме по-рано, на текстовите елементи се задават шрифтове със свойството **font-family**. По подразбиране браузърът използва само шрифтовете, налични в системата. За да има алтернатива, задаваме списък с разделител запетая (,). Имена, които съдържат интервал, се поставят в кавички, напр. 'Times New Roman'.

Пример:

```
p {
  font-family: Tahoma, Verdana, sans-serif;
}
```

Браузърът ще потърси Tahoma, ако не е достъпен – Verdana. Ако и той липсва, ще се използва sans-serif, който е от набора универсални шрифтове, налични във всички браузъри: sans-serif, serif, monospace, cursive, fantasy.

За да сме сигурни, че документът се показва, както очакваме, можем да добавим шрифт, наличен на сървъра, чрез CSS правилото **@font-face**:

```
@font-face {
  font-family: myFont;
  src: url(myfont_file.woff);
}
```

След това използваме във **font-family** както обикновено:

Пример:

```
p {
  font-family: myFont, 'Times New Roman', serif;
}
```

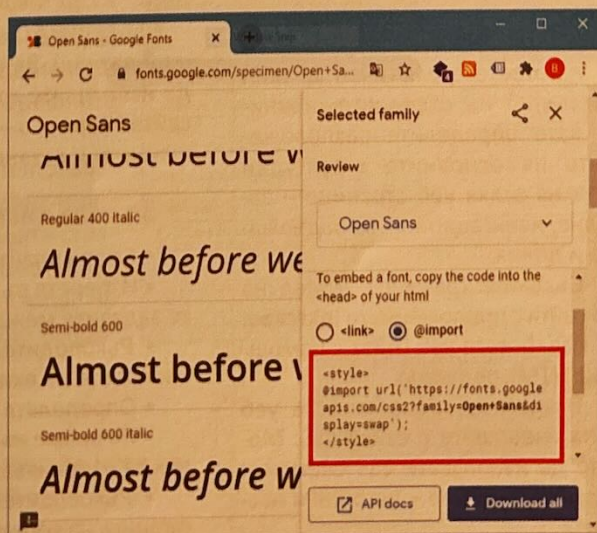
### Уеб услуги за шрифтове

Удобен начин да разнообразим визията на сайта с различни шрифтове са уеб услугите. Една от най-използваните е Google Fonts, предлагаща над 1000 шрифтови семейства. На адрес <https://fonts.google.com/> можем да търсим и разглеждаме, като настройваме различни параметри – категории, езици, размер, стил и др. След като изберем нужните ни начертания, услугата предлага готов код, който да вмъкнем в нашата страница (фиг. 2):

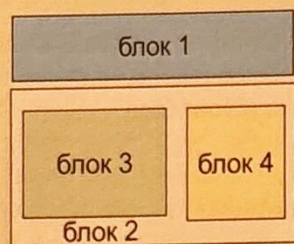
```
<style>
  @import
  url('https://fonts.googleapis.com/css2?
  family=Open+Sans&display=swap');
</style>
```

Показания код с правилото **@import** вмъкваме в секцията **<head>** на HTML документа или, без тага **<style>**, в началото на нашия външен CSS файл. След това можем да използваме шрифта по стандартния начин:

```
font-family: 'Open Sans', sans-serif;
```



Фиг. 2



Фиг. 1



Фиг. 2

### Стъпки за изработване на сайт

- Определете основните изисквания, на които да отговаря уеб сайтът.
- Създайте информационна архитектура на уеб сайта. Определете неговите елементи и изградете навигационната му структура.
- Съберете и обработете информацията, която ще се включва в уеб сайта – текст, графични изображения, мултимедийни продукти.
- Изгответе схематичен дизайн (wireframe) на отделните страници, като определите разположението на основните структурни части на всяка уеб страница – заглавна, навигационна, за съдържание и долна.
- Създайте графичен модел на уеб сайта с приложението **Inkscape**.
- Изработете уеб сайта с помощта на HTML редактор.
- Визуалното оформяне на уеб сайта извършете с езика CSS. Можете да използвате CSS елементи от сайта "Карлово – история и настояще".

Реализацията на даден графичен модел като уеб страница може да се извърши по различни начини и зависи от структурата на графичния модел. Най-универсалният начин се основава на използването на блокове, реализирани най-често с тага `<div>`. С помощта на стилове лесно може да се зададат различни параметри на тези блокове, като размери, цвят, рамка, подравняване и др. Всеки блок от своя страна може да съдържа други блокове с различни параметри. Нека да разгледаме примерната структура на уеб страница от фиг. 1.

За реализирането на HTML кода блоковете се въвеждат последователно в реда на срещането им от горе надолу и от ляво надясно (1, 2, 3, 4), като блок 2 съдържа блок 3 и 4.

За да подредим блоковете 3 и 4 един до друг, ще използваме CSS, като зададем в стила им свойство `"display: inline-block"`, като на десния блок добавим и свойство `"float: right"`. Последните версии на CSS въвеждат по-лесен и мощен модел за подреждане на съдържанието, наречен *Flexbox*. В CSS правилото към контейнера, в случая блок 2, задаваме свойство `"display: flex"`. По подразбиране вложените блокове 3 и 4 ще се подредят в редица от ляво надясно. Начинът и посоката на разполагане и подравняване на елементите в контейнера може да се управлява с допълнителни CSS свойства.

**Задача 1.** Отворете файла `"Site\blocks\blocks.html"` от приложените материали и разгледайте реализирания HTML код на схемата от фиг. 1. Разгледайте файла `"example.html"` с реализация на същата схема с конкретно съдържание.

**Задача 2.** За да добиете представа за възможностите на *Flexbox*, отворете файла `blocks-flex.html` и прочетете коментара за модула *Flexbox*. Вижте в браузъра как се променя уеб страницата, ако задавате различни стойности. Разгледайте файла `"example-flex.html"`, използващ модела *Flexbox*.

На фиг. 2 е дадено подходящо разделяне на блокове на примерен графичен модел.

**Задача 3.** Разгледайте изображението `"Site\blocks\resources\template.jpg"`, което представлява графичен модел на страница, и реализирайте HTML кода му, като спазвате блоковете от фиг. 2. В блоковете добавете само текстовите заглавия и изображенията, които можете да вземете от същата папка.

### Работа по проект

**Вариант 1.** Създайте уеб страниците "Природа" и "Празници" към сайта "Карлово – история и настояще". Създайте графичен модел на новите страници. За реализация на модела можете да използвате страницата "Забележителности" като шаблон, променяйки стилове, за да имат новите страници облик, съответстващ на графичния модел (например заглавието `<h2>` на всяка природна забележителност може да е зелено и др.).

**Вариант 2.** Създайте нов сайт на избрана от вас тема (съгласувана с преподавателя). За улеснение при изготвянето на графичните модели можете да използвате готови шаблони от интернет или да направите шаблони от сайта "Карлово – история и настояще", като изчистите страниците от съдържанието и оставите само основните `<div>` тагове.

### За изпълнение на проекта извършете следните действия:

- Сформирайте екип, който да изпълни зададения проект.
- Изберете ръководител на екипа, изгответе план за работа и разпределете задачите между отделните членове.
- Ръководителят на екипа да създаде група в социална мрежа, в която членовете на екипа да общуват.
- Определете срок за изпълнение на проекта.
- По време на изпълнението на проекта ръководителят на екипа да следи как се изпълняват задачите и дали се спазват сроковете.
- Ръководителят на проекта да води документация, в която да описва процеса на изпълнение на проекта, създадените продукти и начина на работа на членовете.
- Всеки член от екипа да изпълнява в срок поставените задачи.

### Необходимост от тестване на уеб сайт

Тестване на уеб сайт е процес, допълващ неговата изработка и управление. Това гарантира функционалността на изготвения сайт. Прави се проверка на неговото цялостно поведение в работни условия. Това включва:

- Проверка дали дадена уеб страницата е написана според установените стандарти и правила за езика на програмиране (валидиране).
- Тестване на визуализацията и функционалността на уеб сайта, независимо от уеб браузъра и операционната система, която се използва.
- Тестване на скоростта на зареждане и достъпността на уеб сайта.
- Проверка на сигурността на уеб сайта.
- Проверка на администраторската част – контролен панел, достъп и регистрация на потребители, пароли и др.
- Тестването на сайта за мобилна съвместимост с различните видове дисплеи и монитори на различните мобилни и настолни устройства.

### Валидиране на уеб страница

Валидиране на уеб страница е процес на проверка дали страницата е написана според установените стандарти и правила за езика на програмиране. Организацията, която е отговорна за създаването на тези стандарти, е W3C (World Wide Web Consortium). При валидирането се проверява дали всички HTML тагове и техните атрибути са изписани правилно, като са спазени изискванията на приетите стандарти. Правилното изписване на **таговете атрибути** е особено важно, за да се визуализира и функционира сайтът нормално.

Валидацията е един от критериите за качествено изработена уеб страница.

#### Основни предимства на валидирането:

- Лесен за поддръжка код – това намалява времето, прекарано в търсене и отстраняване на грешки в по-късен етап от развитието на сайта.
- Добра съвместимост с бъдещи технологии (нови стандарти/браузъри/програми).
- Бързо добавяне на нова информация.
- Лесно и бързо създаване на нова уеб страница в рамките на валидирания уеб сайт.
- Уеб страницата се зарежда по-бързо от браузъра.
- Валидните сайтове се класират на по-високи позиции при търсене в интернет.

### Инструменти за валидиране на уеб страници

Проверката на валидността на кода на уеб страница може да се проверява по различни начини. Един от тях е писането на HTML и CSS код да се извършва със специализиран софтуер. Тези приложения имат специализирани инструменти, които показват грешките още при писане на кода.

Най-популярният начин за проверка на валидността е използването на т.нар. валидатори. Валидаторът е онлайн инструмент (програма), който ни служи, за да проверим дали кодът на нашата страница е написан според установените стандарти. Валидаторът, който ще използваме, е инструмент, създаден от W3C в помощ на всички уеб разработчици. Той е безплатен и може да бъде използван по всяко време.

#### 1. Валидиране чрез уеб адрес

**Задача 1.** Валидирайте уеб страницата <https://www.mon.bg> чрез валидатора към сайта на W3C на адрес <https://validator.w3.org/>.

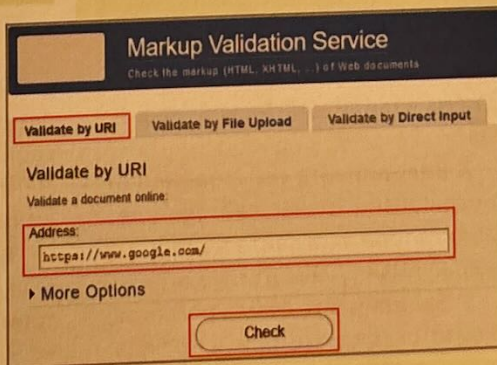
- Отворете страницата: <https://validator.w3.org/>. От горния ляв ъгъл на страницата изберете опцията **Validate by URI** (фиг. 1). Тази опция ви позволява да проверите всяка страница, която е достъпна онлайн.

### Основни понятия

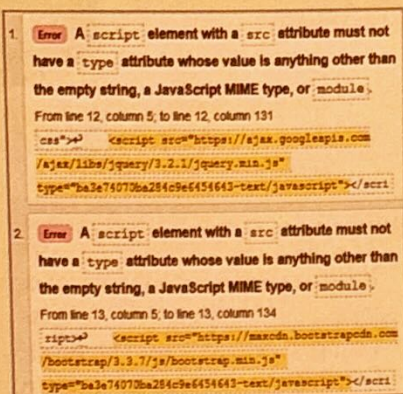
**Валидиране** – процес на проверка дали страницата е написана според установените стандарти и правила за езика на програмиране.

**Емулатор** – инструменти, чрез които се разглежда съвместимостта на различните браузъри и способността на уеб сайт или уеб приложение да функционира в различни браузъри и да се променя, когато функциите на браузъра отсъстват или липсват.

**Тагов атрибут** – атрибут на HTML елемент, описан в неговия таг.



Фиг. 1



Фиг. 2

### Сайтове за тестване:

<https://www.webpagetest.org/>  
<https://gtmetrix.com/>

### Въпроси и допълнителни задачи

1. Какво представлява процесът на валидиране на уеб страница?
2. Валидирайте уеб страници от посещавани от вас сайтове чрез валидатора към сайта на W3C на адрес <https://validator.w3.org/>.
3. Валидирайте файла pois.html, който създадохте при изграждането на сайта "Карлово – история и настояще", чрез валидатора към сайта на W3C на адрес <https://validator.w3.org/>.

- В полето **Address** въведете адреса на страницата, която искате да проверите. След това натиснете бутона **Check**.
- Отваря се страница, в която се появява списък с допуснатите грешки, къде се намират и от какъв тип са (фиг. 2).

## 2. Валидиране чрез качване на файл

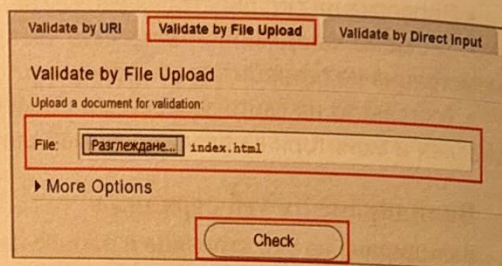
**Задача 2.** Валидирайте файла index.html, който създадохте при изграждането на сайта "Карлово – история и настояще", чрез валидатора към сайта на W3C на адрес <https://validator.w3.org/>.

- Отворете страницата: <https://validator.w3.org/>. От горния ляв ъгъл на страницата изберете опцията **Validate by File Upload** (фиг. 3).

- В полето **File** натиснете бутона **Browse** и от отворения се диалогов прозорец изберете файла, който искате да валидирате.

- В основната страница натиснете бутона **Check**.

- Отваря се страница, в която се появява списък с допуснатите грешки, къде се намират и от какъв тип са.

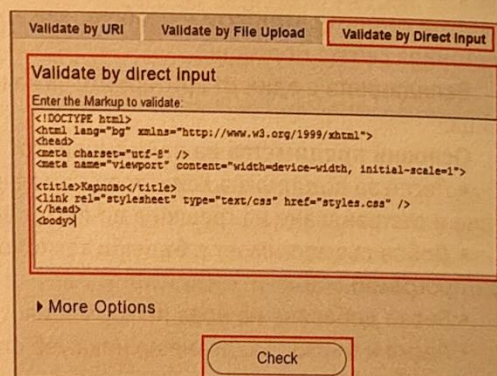


Фиг. 3

## 3. Валидиране чрез директно въвеждане

- Отворете страницата: <https://validator.w3.org/>. От горния ляв ъгъл на страницата изберете опцията **Validate by Direct Input** (фиг. 4).

- В полето **Enter the Markup to validate:** можете директно да напишете кода за валидация. Натиснете бутона **Check**.



Фиг. 4

## Тестване на визуализация на уеб сайт за работа с различни браузъри

При създаване на уеб страница трябва да сте сигурни, че вашите проекти се поддържат от различните браузъри. Почти всеки браузър има свои собствени стилове по подразбиране, поради което някои HTML елементи могат да се показват по различни начини в браузърите, което прави тестването на уеб страниците важно.

За да проверите как изглежда даден уеб сайт в различните браузъри, можете да използвате т.нар. CrossBrowserTesting емулятори. Тези емулятори разглеждат съвместимостта на различните браузъри и способността на уеб сайт или уеб приложение да функционира в различни браузъри и да се променя, когато функциите на браузъра отсъстват или липсват.

**Задача 3.** Тествайте визуализацията на страницата "Забележителности" от уеб сайта "Карлово – история и настояще" с помощта на емулятор. Отворете адреса: <https://www.browserstack.com/>. В сайта имате възможност да тествате визуализацията на създадената от вас уеб страница, като имате безплатен пробен период на ползване.

Достъпността на един сайт определя броя на неговите посетители. Затова той трябва да се вижда еднакво добре от различни устройства, да е независим от операционни системи и браузъри, да се зарежда бързо, да е достъпен за хора с увреждания.

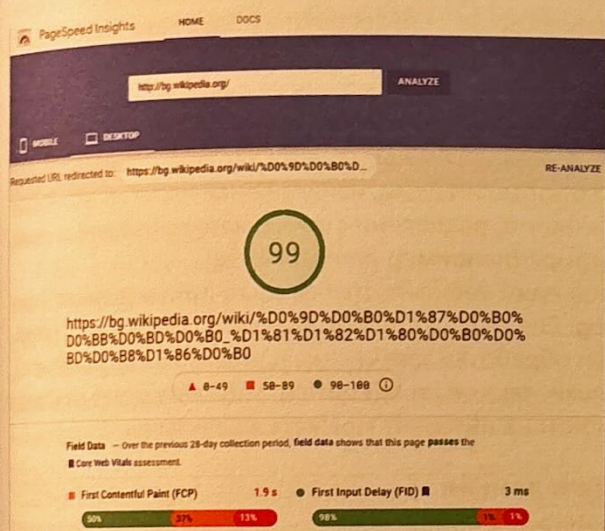
Оптимизацията на сайта спомага за доброто му класиране в търсачките и за бързото му зареждане.

**Задача 1.** Тествайте скоростта и достъпността на уеб страницата <https://bg.wikipedia.org> с помощта на онлайн инструменти.

- Отворете уеб страницата с адрес <https://developers.google.com/speed/pagespeed/insights/>.

- В полето въведете адреса на уеб страницата <https://bg.wikipedia.org> и натиснете бутона **Analyze** (фиг. 1).

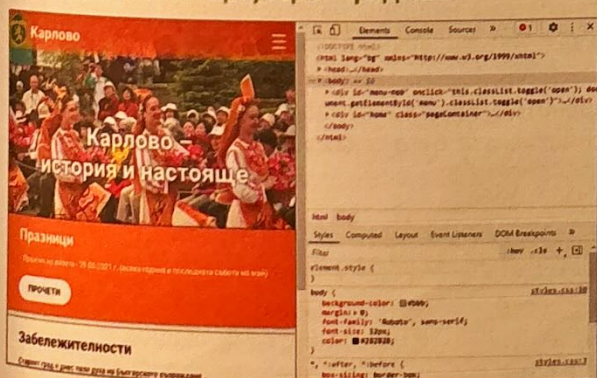
- В сайта се изписват различни параметри на тестваната от вас уеб страница. Можете да видите данни за мобилната и настолната версия на страницата.



Фиг. 1

Тестване на различните параметри на уеб страницата може да се извършва и чрез уеб инструментите на браузърите. В зависимост от браузъра в неговото главно меню има меню, което съдържа различни опции за тестване на параметрите на уеб страницата.

**Задача 2.** Заредете създадена от вас уеб страница от сайта "Карлово – история и настояще" с уеб браузъра Google Chrome. Разгледайте и тествайте различни нейни характеристики с помощта на инструментите, които браузърът предлага.



Фиг. 2

- Заредете браузъра Google Chrome.
- Отворете главното меню от трите хоризонтални линии, разположени в горния десен ъгъл на прозореца на браузъра, и изберете менюто **Още инструменти/Инструменти за програмисти (More tools/Developer tools)** или натиснете клавиша F12.

- В дясната част на екрана на прозореца се визуализират различните инструменти, а в лявата част е визуализирана страницата (фиг. 2).

## Инструменти за анализ:

- Елементи (Elements) – предоставя ви възможност да прегледате CSS и HTML кода, както и да ги редактирате в движение, виждайки ефектите на вашите промени в реално време.

- Конзола (Console) – JavaScript конзолата позволява директно въвеждане на команди, както и отстраняване на грешки.

- Източници (Sources) – позволява отстраняване на грешки при JavaScript код чрез мощен графичен интерфейс.

- Мрежа (Network) – категоризира и показва подробна информация за всяка свързана операция в страницата, включително пълните заглавия на заявките и отговорите.

- Производителност (Performance) – предоставя ви възможност да тествате ефективността на сайта, като наблюдавате времената за зареждане. Можете също да симулирате различни обстоятелства, като например мобилни процесорни възможности.

- Памет (Memory) – предоставя ви възможност да наблюдавате и да записвате използването на паметта на уеб страницата.

- Сигурност (Security) – отбелязва проблеми със сертификатите и други проблеми, свързани със сигурността.

- Приложение (Application) – прави проверка на ресурсите, използвани от уеб приложение. Можете да получите пълна разбивка на това, което се използва.

- Одити (Lighthouse) – предлага начини за оптимизиране на времето за зареждане на страницата и показва общата производителност.

**Режимът на устройството (Device mode)** ви позволява да видите активната страница в симулатор почти точно така, както би се появила на различни мобилни устройства с операционни системи Android и iOS. Също така имате възможност да избирате различни резолюции на екрана. За да включите и изключите **Device Mode**, изберете иконата на мобилния телефон, разположена директно вляво от раздела **Elements**.

**Задача 3.** Заредете някои от любимите си сайтове с уеб браузъра Google Chrome. Разгледайте и тествайте неговите характеристики с помощта на инструментите, които браузърът предлага.

**Задача 4.** Тествайте скоростта и достъпността на създадените от вас уеб страници от сайта "Карлово – история и настояще" онлайн на адрес <https://developers.google.com/speed/pagespeed/insights>.

Публикуването на уеб сайт включва: избор на организация, която предлага услугата уеб хостинг, избор на домейн на уеб сайта, същинско публикуване и управление на съдържанието на сайта с уеб сървър.

**Уеб хостинг** (англ. web hosting) е услуга, която позволява да качите сай-

та си на сървър (хост), така че да е видим за посетителите. Обикновено хостингът е споделен – на сървъра има много потребители, като всеки от тях е наел малка част от ресурсите на сървъра за определен период от време.

При избор на уеб хостинг е важно да се знае дали услугите са безплатни, или са платени. Когато търсите безплатен уеб хост, трябва да внимавате. Някои доставчици на безплатен уеб хостинг ще качват на сайта ви много реклами. Други недостатъци са ограничено дисково пространство за съх-

ранение, по-ниска скорост и проблеми с производителността. Предимство имат доставчици на хостинг услуги, които предлагат инсталиране на системи за управление на съдържанието (CMS), като предоставят лесен за управление контролен панел.

**Домейн** (англ. domain) е името, с което ще се вижда вашият уеб сайт, когато бъде публикуван в интернет. В интернет не могат да съществуват два напълно еднакви домейна (имена на уеб сайтове). Имената на домейните се образуват от отделни думи, изписани най-често на латиница, без интервал между знаците, разделени с точки, като се използват само малки букви, цифри и тире (например romagallnik.bg).

За да бъде достъпен един уеб сайт, трябва да го публикувате на компютър, наречен **сървър**. Специална програма, наречена уеб сървър (англ. web server), получава и обработва заявки, идващи от клиенти (браузъри), и отговаря на тези заявки, така че страниците от уеб сайта могат да бъдат видени от всички клиенти на интернет мрежата в браузъра.

### Избор на хостинг и домейн

**Задача 1.** Регистрирайте хостинг акаунт и домейн, като използвате услугите на безплатен уеб хостинг.

За публикуването на уеб сайта "Карлово – история и настояще", който създадохме в предишните уроци, ще използваме услугите на безплатен уеб хостинг, които се предлагат на сайта **infinityfree.net**.

Процесът на регистрация преминава през няколко стъпки:

- Заредете уеб страницата **infinityfree.net** и изберете бутона за регистрация **Sign Up Now** (фиг. 1).

- Отваря се прозорец за регистрация в сайта (фиг. 2), в който въвеждате действащ имейл адрес и парола. Поставете отметки в полетата **I've read and agree to the terms of service** и **I'm not a robot**, с което потвърждавате, че сте запознати с общите правила за ползване на услугите на сайта, и натиснете бутона **Create new account**.

- Регистрационната система изпраща на посочения от вас имейл писмо, в което има бутон за потвърждаване на въведения във формата за регистрация имейл.

- След потвърждаването в имейла на успешната регистрация се отваря прозорец, в който натискате бутона **Create account**, за да създадете свой хостинг акаунт.



Фиг. 1

Фиг. 2

Фиг. 3

Фиг. 4

### Сайтове, доставящи безплатни уеб хостинг услуги:

[www.freehosting.com/](http://www.freehosting.com/)  
[googiehost.com/](http://googiehost.com/)  
[www.awardspace.com/](http://www.awardspace.com/)

[www.000webhost.com/](http://www.000webhost.com/)  
<https://byet.host/>  
[freehostingnoads.net/](http://freehostingnoads.net/)

- В прозореца **Create a Hosting Account** (фиг. 3) изберете тип и име на домейн и натиснете бутона **Search Domain**. За тип на домейн изберете опцията **Subdomain** (поддомейн).

- Отваря се прозорец (фиг. 4), в който задавате парола за достъп до хостинг акаунта и натискате бутона **Create account**. Системата автоматично генерира потребителско име на акаунта.

- Отваря се прозорец (фиг. 5), в който се съдържа информация за създадения акаунт.

**Задача 2.** Публикувайте уеб сайта "Карлово – история и настояще" на уеб хостинга, на който се регистрира в задача 1.

Зареждането на уеб хостинга се извършва от уеб страницата **app.infinityfree.net**. За да влезете в хостинг акаунта, трябва да въведете имейл и парола.

Управлението на уеб хостинга се извършва през контролния му панел – **Control Panel** (фиг. 6), на който се намират различни инструменти за управление на файлове и база данни, статистически данни и др.

Публикуването на данни можете да извършите директно чрез инструмента **Online File Manager** или чрез FTP клиент.

- За да отворите файловия мениджър на уеб хостинга, изберете **Online File Manager**. Отваря се прозорец (фиг. 7), в който се намират папките и файловете на хостинга, както и инструменти, чрез които се извършват различни операции.

- От файловата система изберете папката **htdocs**, тъй като файловете, които участват в изграждането на уеб сайта, задължително трябва да се качват в нея.

- Качването на файлове (папки) стартирайте от бутон **Upload/Upload File (Folder)** и от файловата система на вашия компютър изберете желаните файлове.

## Правни и етични норми при публикуване на информация в интернет

При публикуване на информация в интернет трябва да се спазват **определени правни и етични норми**:

- Съдържанието на сайта не трябва да съдържа материали, които са обект на авторско право, или, ако има такива, те да се използват с разрешение на авторите.

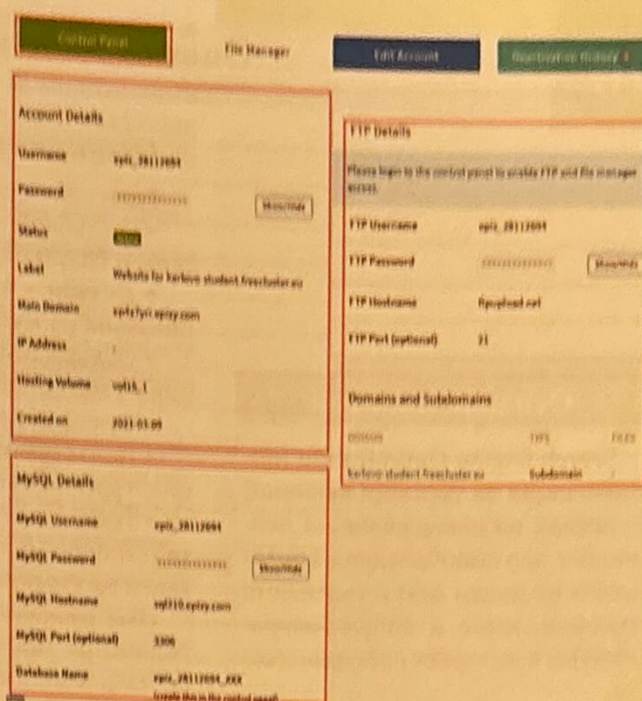
- Коректно да се цитират източниците на информация. Когато се използва готов текст от книга или друг текстов източник, той трябва да се постави в кавички и да се посочат неговият автор и източникът, от който е взета тази информация. При използване на информация от уеб сайт може да се посочи хипервръзката към него.

- Информацията в сайта трябва да отговаря на истината, тъй като авторът носи отговорност за нея.

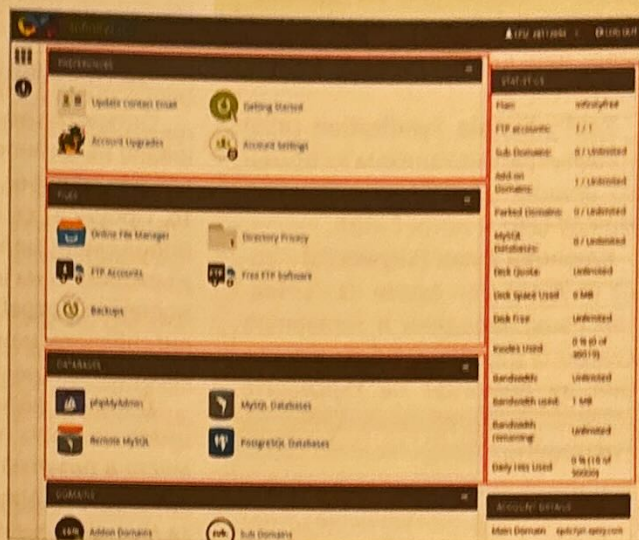
- Съдържанието не трябва да е нецензурно.

- В сайта не трябва да се съдържат заплахи или обиди, да се накърняват правата и достойнствата на други лица.

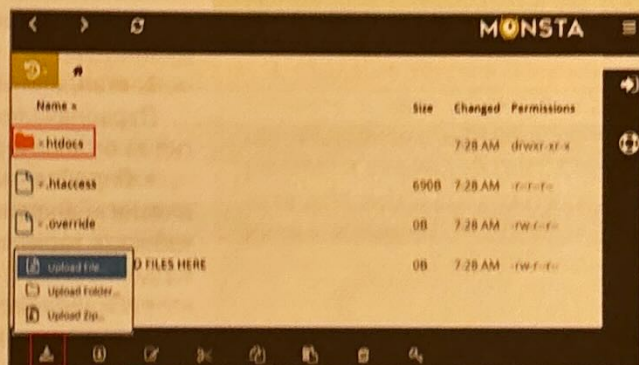
- В материалите не трябва да се подбужда към расова, етническа или верска омраза, към престъпления или нарушаване на закона.



Фиг. 5



Фиг. 6



Фиг. 7

## Въпроси и допълнителни задачи

1. Какво представлява процесът на валидиране на уеб страница?
2. Разгледайте сайта **free.bg**, който предоставя безплатни хостинг услуги. Ако имате желание, регистрирайте домейн с избрано от вас име. Обърнете внимание на опциите, които той предоставя – големината на качваните файлове, общо дисково пространство.

### Основни понятия

#### Search Engine Optimization (оптимизация за търсещи машини)

– процес на извършване на дейности с цел подобряване на позицията на даден сайт в търсещите машини, както и популяризирането му в интернет пространство.

**Рангово подреждане (ранжиране)** – подреждане на оценяваните сайтове в реда на подобряване или влошаване на техните качества.

**Really Simple Syndication (RSS)** – софтуерен механизъм за обмен на новини между два сайта или между сайт и потребител.

**Ключови думи (keywords)** – това са думи или фрази на съответен език, зададени в заглавната секция (head) на уеб страница с цел да помогнат на търсещите машини в категоризирането на страницата/сайта.

### Оценка на уеб сайт

Един сайт може да се оцени по различни показатели, например:

- **Достъпност** – брой на посетителите, независимост от операционни системи и браузъри, скорост на зареждане, достъпност за хора с увреждания.
- **Дизайн** – визуално представяне, привлекателност, уникалност, използване на мултимедия, лесен за навигация.
- **Съдържание** – точност, обективност, актуалност, качество и адаптация на материалите.

- **Функционалност** – модули за работа с информацията, обратна връзка, търсене, интерфейс, форми, помощ.
- **Ползваемост** – ергономичност и удобство на използването, навигация, търсене, интерфейс, форми, помощ.

- **Технология** – спазване на стандарти при използване на съвременните технологични средства (аудио, видео, социални мрежи и др.), оптимизация на страниците и графиките.

Има различни платформи, които посредством оценяване на сходни параметри могат да дадат комплексна оценка (ранг) на един сайт – например [www.alexa.com](http://www.alexa.com), [www.site.info](http://www.site.info), и др.

**Задача.** Вижте кои са първите 50 сайта с най-висок ранг за България.

За целта влезте в сайта [www.alexa.com](http://www.alexa.com) и от менюто "Tools->Top Sites" можете да получите нужната информация.

### Оптимизация на уеб сайт

Оптимизацията е систематизиран и продължителен процес на извършване на дейности с цел подобряване на позицията на даден сайт в търсещите машини, както и популяризирането му в интернет пространството. Процесът се нарича още **SEO оптимизация** и включва извършване на определени целенасочени промени по кода, структурата, дизайна и съдържанието на уеб сайта, така че да го направи по-привлекателен и стойностен за потребителите и търсачката. SEO е термин, който произлиза от английското наименование Search Engine Optimization (оптимизация за търсещи машини).

Добрата оптимизация ще повлияе положително върху популяризацията на сайта, което означава, че той ще се позиционира на по-високи места в резултатите при търсене по ключови думи. Така той ще бъде видим за по-обширна интернет аудитория, което ще увеличи посещаемостта му и ще го направи по-популярен и известен.

### Популяризиране на уеб сайт

Популяризирането на уеб сайта преминава през различни оптимизационни етапи.

#### 1. етап. Анализ, проучване и планиране.

Първоначален анализ на уеб сайта и изграждане на цялостна стратегия за оптимизирането му.

- **Определяне на потребителската аудитория.** Определяне, уточняване и анализиране на аудиторията, към която ще бъде насочен уеб сайтът, в зависимост от неговата тематика, вида на услугите и характера на дейността.

- **Изследване на конкуренцията.** Проучване за представянето на най-високо класираните сходни сайтове в търсачките, за да се формира ефективна стратегия, която ще бъде използвана за постигане на желаното позициониране на вашия уеб сайт спрямо тях.

- **Избор на ключови думи.** Ключовите думи трябва да са внимателно подбрани въз основа на честотата на търсенето им в интернет. Подходящ инструмент за проверка на тази честота е Google Trends ([trends.google.com](http://trends.google.com)). Броят на ключовите думи не трябва да е много голям. Въпреки че напоследък търсачките анализират самото съдържание на страниците и по този начин ги категоризират, ключовите думи все още са важни.

#### 2. етап. Вътрешна SEO оптимизация.

- **Инспекция на съдържанието.** Преглед на количеството и качеството на текстовото съдържание във всяка една от страниците – доколко то

е уникално, стилизирано, коректно написано и подредено. Обогаляване на съдържанието с нова допълнителна информация с цел да се постигне максимално подробно обяснение на темата. Изследване за евентуалното наличие на дублираща се информация и текстове в параграфи и абзаци. Тестване за правилната употреба и оптималната честота за наситеност на ключовите думи в отделните текстове.

- **Проверка на изходния код.** Анализиране на сайта с цел да се гарантира, че той има добър изчистен HTML код и работи нормално. Проверка и валидиране на кода с помощта на валидатор.

- **Мобилна съвместимост.** Мобилната съвместимост на всеки уеб сайт е от съществено значение и в момента тя е с приоритет за търсещите машини, тъй като все по-голям брой потребители използват мобилните си телефони.

- **Скорост на зареждане.** Предприемат се технически стъпки за ускоряване на зареждането на всяка страница чрез прилагането на специални методи за компресиране на HTML кода, отложено зареждане на изображенията (Lazy Load), минимизиране и отлагане на зареждането на файлове.

- **Използване на метатагове.** Формират се и се попълват най-важните за търсачките метаданни в кода. Прави се корекция на метатаговете за заглавия и описания на базата на текстовото съдържание и критериите за откриваемост. Прави се проверка за лимит на брой символи в заглавните и описателните метатагове.

- **Използване на заглавни тагове.** Тества се и се анализира заглавната HTML структура и се коригират заглавните тагове, които определят структурата на съдържанието в страницата и указват нейния смисъл и тематика. Търсещите машини засичат и разчитат тези тагове, като по този начин разбират за какво се отнася написаното в нея. Най-важният за оптимизацията от всички тях е H1 тагът, който представлява и основното заглавие в страницата.

- **Структура на връзките.** Изграждане на правилна и коректна мрежа от вътрешни връзки в уеб сайта – хиперлинкове с текстове, които свързват страниците помежду им. Наличие на вътрешна навигация – меню с бутони към най-важните линкове в сайта.

- **Създаване на карта на уеб сайт (Sitemap).** Създаване на специален файл с карта на уеб адресите на всички уеб страници, които са предназначени за индексирание. Файлът ориентира търсещите машини за всички вътрешни линкове в уеб сайта и указва адреса на всяка страница, която трябва да бъде индексирана.

### 3. етап. Външна SEO оптимизация.

Външната оптимизация включва регистрация в търсачки, социални мрежи, каталози и анализ на съществуващите обратни връзки. Изграждат се нови външни линкове от други уеб сайтове с добър рейтинг както към началната страница, така и към определени вътрешни страници. Използват се директории за писане на статии, блогове с коментари, бизнес директории, социални мрежи, форуми и др. Извършва се интегриране на бутони за споделяне на страници, услуги, продукти и публикации в социалните мрежи.

### 4. етап. Периодичен отчет за трафик и класиране.

Периодично се проследява трафикът от външни източници. Извършва се отчет за изследване на промените в класирането за най-силните ключови думи и изрази. Изготвят се допълнителни технически промени и изменения за подобряване представянето на отделните страници. Допълнително се обогатява и подобрява информацията с нов полезен текст, изображения, видео- и аудиоклипове, за да може тя да стане по-достъпна за потребителя.

Популяризиране на уеб сайт може да се извърши и с помощта на специален софтуерен механизъм за обмен на новини между два сайта или между сайт и потребител, така наречения RSS (англ. Really Simple Syndication). Представлява набор от формати за захранване с информация от интернет. Използва се от приложения или онлайн услуги, наречени агрегатори, които могат автоматично да получават и показват последните новини от произволен брой информационни канали. Технологиите позволяват на всеки желаещ да се абонира и да получава едновременно информация от стотици сайтове, без те да бъдат директно посещавани. За ползване на услугата е необходимо инсталиране на RSS четец.

## Въпроси и задачи

1. Кои са основните критерии, които включва процесът по оценка на един сайт?
2. Какво представлява процесът на SEO оптимизация на уеб сайт?
3. Създайте текстов документ и опишете в него мерките, които ще предприемете за популяризирането на вашия уеб сайт. Споделете документа със своите съученици в интернет.

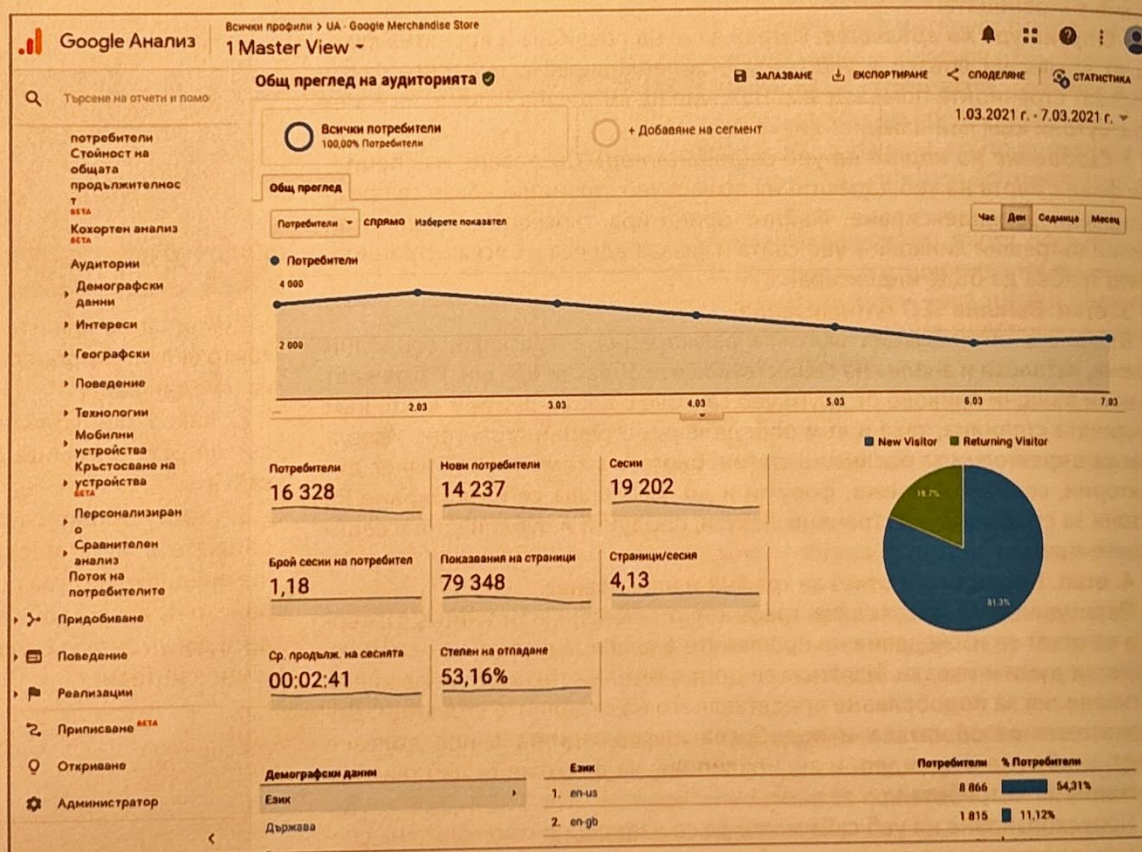
## Инструменти за анализ на посещаемостта на уеб сайт

Инструментите за уеб анализ са тези, чрез които се преценяват успехите и неуспехите на всеки уеб сайт. Ако се ползват правилно, информацията, която те предоставят, помага за оптимизирането на сайта. Да се прави анализ на ключови думи, източници на трафик, на посещения, начин на откриване, е изключително важно, тъй като това помага да бъдат предприети действия за подобряване на дадения сайт и неговото позициониране.

Чрез анализа на трафика може да се проследят тенденциите в поведението на различни групи от потребителите на сайта. Благодарение на инструментите за анализ на трафика могат да се вземат решения за промени в изгледа, структурата и съдържанието на уеб сайта.

Един от най-добрите инструменти за анализ на посещаемостта е **Google Analytics** (Гугъл анализ). Това е безплатна услуга, предоставена от Google, но за да я използвате, е нужно да имате администраторски достъп до сайта за анализиране. За да е достъпен все пак инструментът за изучаване, Google предоставя достъп с помощта на демонстрационен акаунт/demo account.

**Задача 1.** Отворете уеб сайта <https://support.google.com/analytics> и се запознайте с услугата Google Analytics. В края на страницата има връзка към демонстрационния акаунт. За да го използвате, трябва да сте се вписали в Google с ваш профил. Последвайте връзката и влезте на страницата на Google Analytics, за да разгледате предлаганите инструменти (фиг. 1).

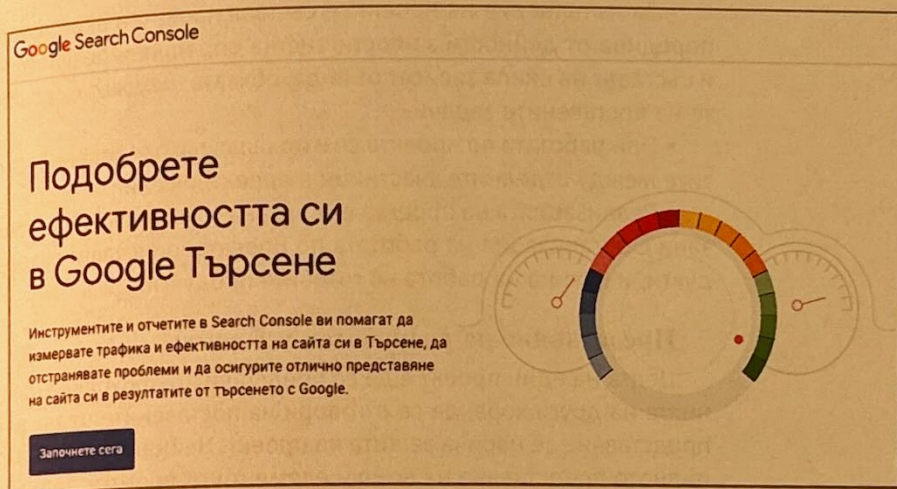


Фиг. 1

Съществуват различни инструменти, които помагат за първоначалното индексване на уеб сайта и измерването на трафика и ефективността в него. Те отчитат наличието на евентуални технически проблеми и предлагат методи за отстраняването им. Такава услуга е **Google Search Console** (Гугъл конзола за търсене). Тя включва инструменти, които позволяват на уеб администраторите да извършват определени действия за оптимизирането на уеб сайта, като:

- Да проверяват колко пъти сайтът е излизал в резултата от търсене и колко пъти е посетен.
- Да проверяват страниците, от които има връзка към сайта.
- Да проверяват страници с ключови думи и извършен достъп до сайта.
- Да виждат чрез какви търсения с ключови думи е извършен достъп до сайта.
- Да проверяват дали сайтът е пригоден за показване на мобилни устройства.
- Да проверяват за проблеми със сигурността на сайта.

**Задача 2.** Отворете уеб сайта <https://search.google.com/search-console/about> и се запознайте с възможностите на Google Search Console (фиг. 2).



Фиг. 2

## Методики за оценяване на уеб сайт

Съществуват различни методики за оценяване на уеб сайтове.

**Тестване за ползваемост (usability test)** – извършва се с потребители на сайтове. Проучването разкрива проблемите, които хората срещат при ползване на сайт. По време на тестовите се наблюдава как потребителите ползват сайт в естествена среда и се разговаря с тях. Така се научава какво харесват, какво ги затруднява, какво още искат да се добави в сайта и кое е излишно. В резултат се получават препоръки за подобряване на началната страница, графичния дизайн, навигацията, структурата, функционалността и съдържанието на сайта.

**Оценка на информационните ресурси.** Това е важно, тъй като много ресурси не се проверяват за точност, а вие носите отговорност за тяхната валидност. Проверявайте вашите данни, критично анализирайте различните текстове. Основните критерии, по които се оценяват информационните ресурси, са:

- Уместност (дали информацията има отношение към разглеждания въпрос).
- Приложимост (дали информацията е подходяща за моята възраст и основни ценности).
- Детайлност (колко информация ми трябва).
- Актуалност (кога беше публикувана или последно актуализирана информацията).
- Достоверност (кой е авторът на информацията).
- Пристрастност (защо беше написана тази информация).

**Задача 3.** Направете оценка на ранга на уеб сайт по избор. Използвайте сайта [www.alexa.com](http://www.alexa.com). Информацията за конкретен сайт е достъпна от менюто "Tools->Web Site Traffic Statistic". Имайте предвид, че ако сайтът е нов, може да не са се натрупали достатъчно данни за неговата оценка. Намерете къде в излязлата информация има данни за подобни сайтове. Вижете и техния ранг.

### Принципи на управление на проекти

Управлението на проекти се базира на следните основни принципи:

- Всеки проект трябва да решава конкретен проблем, който предизвиква определен интерес и е съобразен с поставената цел.
- Проектът се разработва на етапи. Обикновено всеки следващ етап на проекта представлява повторението на даден процес към крайната цел и надгражда предишния етап, добавяйки нова функционалност към крайния резултат от предишния етап.
- Изпълнението на проекта може да доведе до уникални резултати и той да бъде полезен за определен брой хора.
- При реализацията на проект има изисквания към вида и качеството на получените резултати.
- За изпълнение на проект се съставя проектен екип, който извършва поредица от дейности за постигане на определен резултат. Числеността и съставът на екипа зависят от вида, обхвата, бюджета, сроковете и целите на поставените задачи.
- При работата по проекта се извършва разпределение на отговорностите между отделните участници в проектния екип.
- Реализацията на проекта е свързана с изготвяне на документи, свързани с организация на работата по проекта, описващи създадените продукти, и начина на работа на готовите продукти.

### Представяне на разработен уеб сайт

Целта на един проект е да се представи пред публика, да се чуят мненията на други хора, да се отговори на поставените от тях въпроси. Това представяне се нарича защита на проект. Нейната роля се изразява в пълното представяне на всички елементи от проекта, задълбочена обясновка на взетите решения, описание на цялостния процес на работа и срещнатите проблеми при изпълнението.

**Задача 1.** Представете пред съучениците си проекта на сайта, който изготвихте. По време на представянето опишете:

- екипа и разпределението на задълженията за осъществяване на проекта;
- основните етапи, през които преминахте при изработването на сайта, и проблемите, които срещнахте по време на работата си;
- неговата навигационна структура и графичен дизайн.

### Инструменти и стъпки за оценка на уеб сайт

**Задача 2.** Представете пред съучениците си стъпките, чрез които ще извършите оценка на вашия уеб сайт. По време на представянето обърнете внимание на:

- съпоставяне на сайта с конкурентни сайтове;
- установяване на слабите места на сайта;
- предоставяне на препоръки за модернизирането на сайта;
- извършване на тест за ползваемост.

### Оценяване на уеб сайт

**Задача 3.** Представете пред съучениците си резултати от тестове, които сте провели:

- за валидиране на отделните страници;
- за визуализация и функционалност на уеб сайта в различните уеб браузъри;
- за скоростта на зареждане на уеб сайта;
- за проверка на сигурността на уеб сайта.

## 1. HTTP е:

- А) протокол, който се използва за обмен на хипертекстове
- Б) метод за предаване на информация, при който части от текст, изображение, звук и др. могат да съдържат връзка към друга информация
- В) начин на организация на работа и обмен на данни в интернет
- Г) нормативни спецификации на технологии и методологии за визуализиране на съдържание в уеб среда

## 2. Мястото, на което са разположени уеб страниците, групирани в един уеб сайт, се нарича:

- А) уеб стандарт
- Б) хипертекст
- В) HTTP
- Г) уеб сървър

## 3. Какво представлява World Wide Web Consortium (W3C):

- А) стандарт на интерфейс, предназначен за съвместна работа на уеб сървър и интерпретаторите на скриптове
- Б) език за описание и дизайн на уеб страници
- В) система за управление на съдържанието
- Г) организация за стандартизация в интернет

## 4. Основният маркиращ език за описание и дизайн на уеб страници е:

- А) HTML
- Б) JavaScript
- В) CSS
- Г) PHP

## 5. Кой от посочените тагове в HTML се използва за включване на основно съдържание:

- А) <body>
- Б) <title>
- В) <head>
- Г) <html>

## 6. Тагът head:

- А) съдържа основното съдържание на документа
- Б) отбелязва началото и края на документа и съдържа всички други негови елементи
- В) съдържа заглавието на документа
- Г) съдържа заглавната част на файла

## 7. Кой от посочените тагове в HTML се използва за създаване на хипервръзка:

- А) <p>
- Б) <title>
- В) <a>
- Г) <h6>

## 8. Кой от посочените тагове в HTML се използва за вмъкване на изображения:

- А) <tr>
- Б) <img>
- В) <a>
- Г) <br>

## 9. Процесът на проверка дали страницата е написана според установените стандарти и правила за езика на програмиране се нарича:

- А) тестване
- Б) валидиране
- В) оптимизация
- Г) достъпност

## 10. Оптимизацията е:

- А) извършване на дейности с цел подобряване на позицията на даден сайт в търсещите машини, както и популяризирането му в интернет пространството
- Б) подреждане на оценяваните сайтове в реда на подобряване или влошаване на техните качества
- В) проверка дали страницата е написана според установените стандарти и правила за езика на програмиране
- Г) документиране ефикасността на един сайт

## 11. Софтуерен механизъм за обмен на новини между два сайта или между сайт и потребител се нарича:

- А) Search Engine Optimization (SEO)
- Б) ранжиране
- В) Really Simple Syndication (RSS)
- Г) система за управление на съдържанието (CMS)